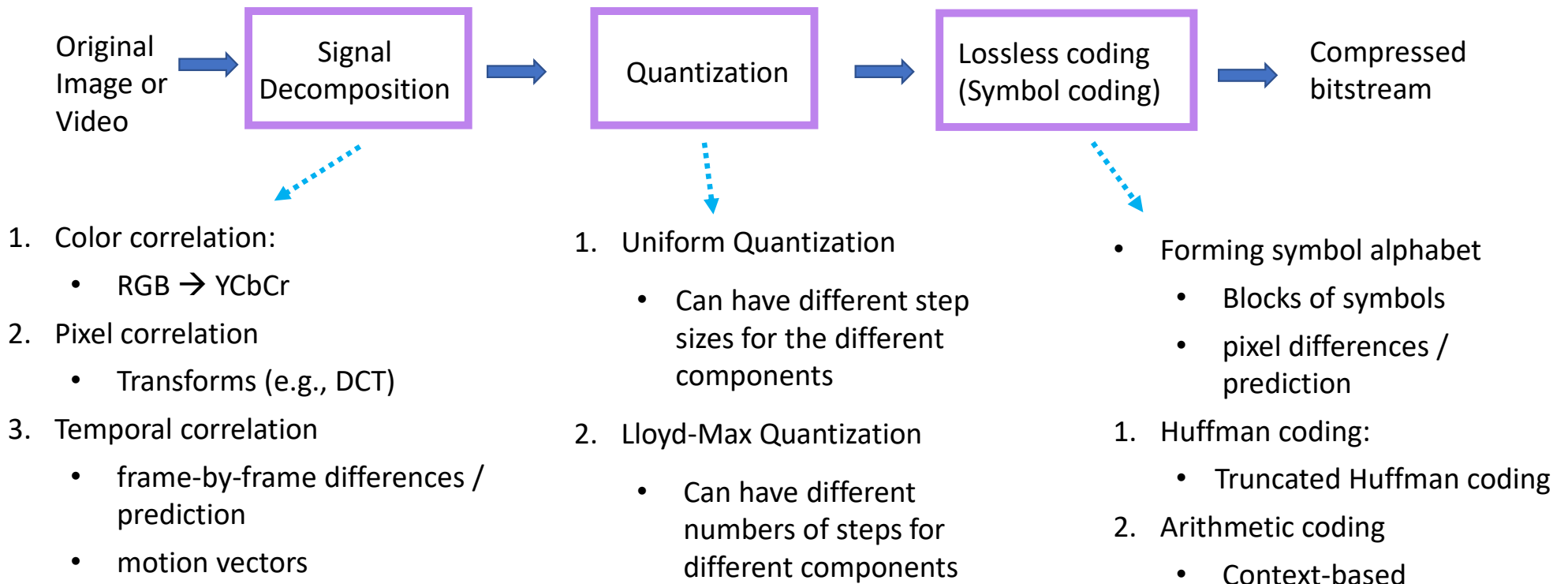


ECE 253 – Digital Image Processing

DCT, JPEG, Predictive Coding

General System for Image/Video Compression



Discrete Cosine Transform: Definition

The 1-dimensional (1-D) N-point DCT is defined by:

$$C(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \quad \text{for } u=0,1,2,\dots,N-1.$$

where

$$\alpha(u) = \begin{cases} \sqrt{1/N} & \text{for } u = 0 \\ \sqrt{2/N} & \text{for } u = 1,2,\dots,N-1 \end{cases}$$

The inverse DCT is defined by

$$f(x) = \sum_{u=0}^{N-1} \alpha(u) C(u) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \quad \text{for } x=0,1,2,\dots,N-1.$$

N signal values
(real valued)



N transform coefficients
(real valued)

2-DCT

- The DCT is linear and reversible
- The 2-D transform is a separable product of 1-D transforms:

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left[\frac{(2x+1)u\pi}{2N}\right] \cos\left[\frac{(2y+1)v\pi}{2N}\right]$$

for $u, v = 0, 1, 2, \dots, N-1$.

The inverse transform is:

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u)\alpha(v) C(u, v) \cos\left[\frac{(2x+1)u\pi}{2N}\right] \cos\left[\frac{(2y+1)v\pi}{2N}\right] \quad \text{for } x, y = 0, 1, 2, \dots, N-1.$$

2D DCT Basis Functions

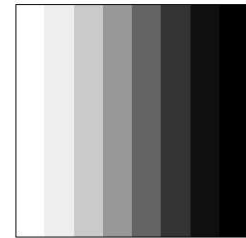
- Suppose $N=8$. Consider taking just one non-zero coefficient in this inverse transform equation:

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u) \alpha(v) C(u, v) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right]$$

- Say $C(1,0) = 1$, all other coefficients are zero

$$f(x, y) = \alpha(1) \alpha(0) C(1,0) \cos \left[\frac{(2x+1)\pi}{2 \times 8} \right] = \sqrt{\frac{1}{8}} \sqrt{\frac{2}{8}} \cos \left[\frac{(2x+1)\pi}{2 \times 8} \right]$$

$$f(x, y) = \frac{\sqrt{2}}{8} \cos \left[\frac{(2x+1)\pi}{16} \right] \quad \text{for } x, y = 0, 1, \dots, 7.$$



2D DCT Basis Functions

Matlab: dct2 and idct2

$$C(1,0) = 1$$

$$f(x,y) = \frac{\sqrt{2}}{8} \cos\left[\frac{(2x+1)\pi}{16}\right]$$

```
>> idct2(DCTarray)
```

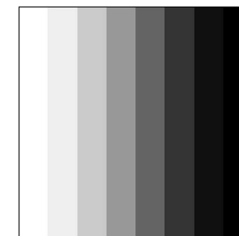
```
ans =
```

```
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
0.1734    0.1470    0.0982    0.0345   -0.0345   -0.0982   -0.1470   -0.1734
```

```
DCTarray =
```

```
0    1    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
0    0    0    0    0    0    0    0
```

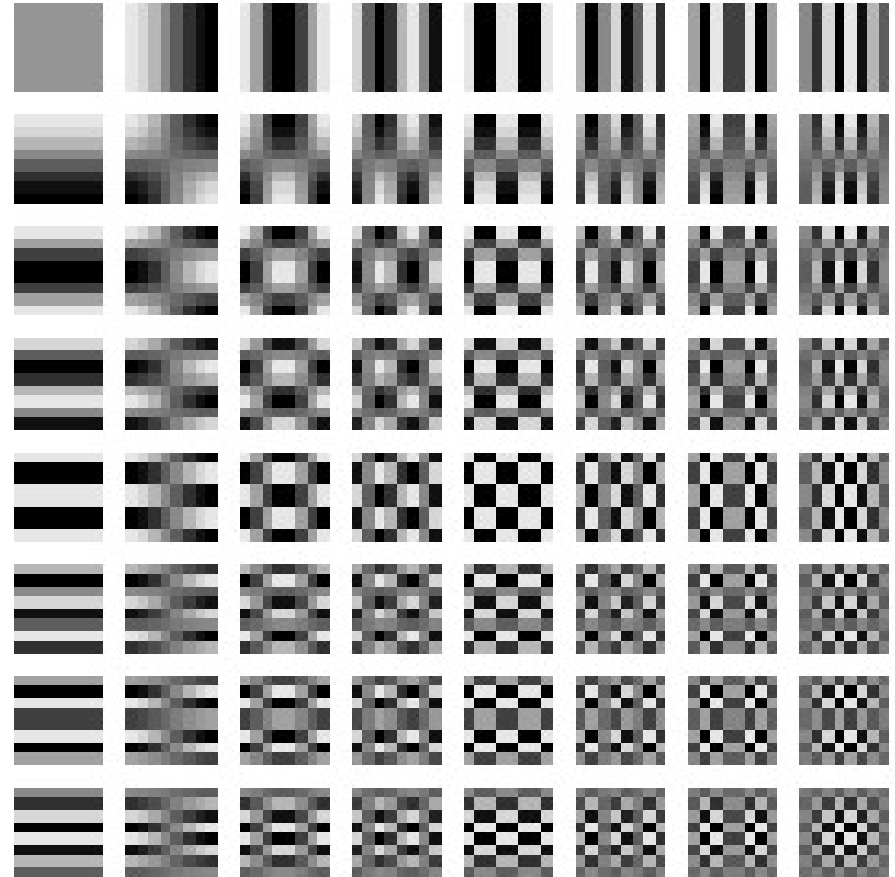
Shift everything to be positive for display
Rescale to the range 0→1
Imshow:



2D DCT Basis Functions

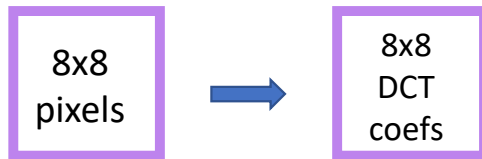
The whole array of basis functions looks like this:

The DCT coefficients tell us how much of each of these basis images we need to regenerate original block $f(x,y)$

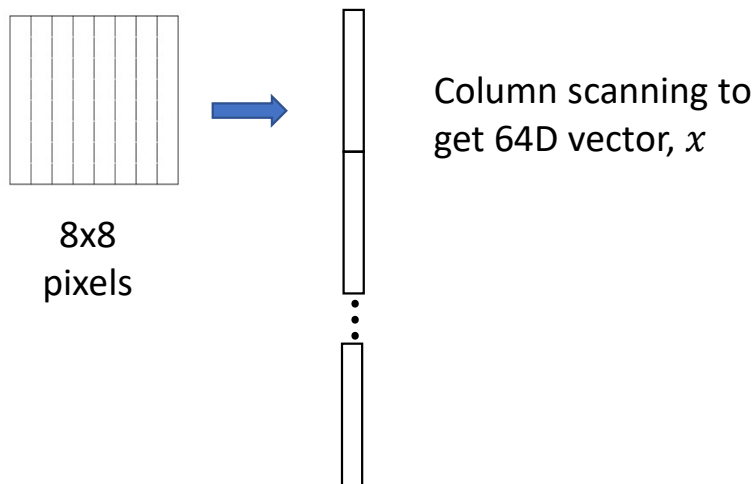


$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u) \alpha(v) C(u, v) \cos\left[\frac{(2x+1)u\pi}{2N}\right] \cos\left[\frac{(2y+1)v\pi}{2N}\right]$$

DCT can be implemented as a unitary transform



One way to think of this is as:



$$y = T x$$

Diagram illustrating the equation $y = T x$:

- y : Coefficient vector of length 64
- T : 64 x 64 matrix which implements the 2D DCT
- x : Pixel vector of length 64

T is a unitary matrix:

$$T^{*t} = T^{-1}$$

Unitary transforms are distance preserving

Let x, x', y, y' be vectors of length N

Let T be a unitary $N \times N$ matrix

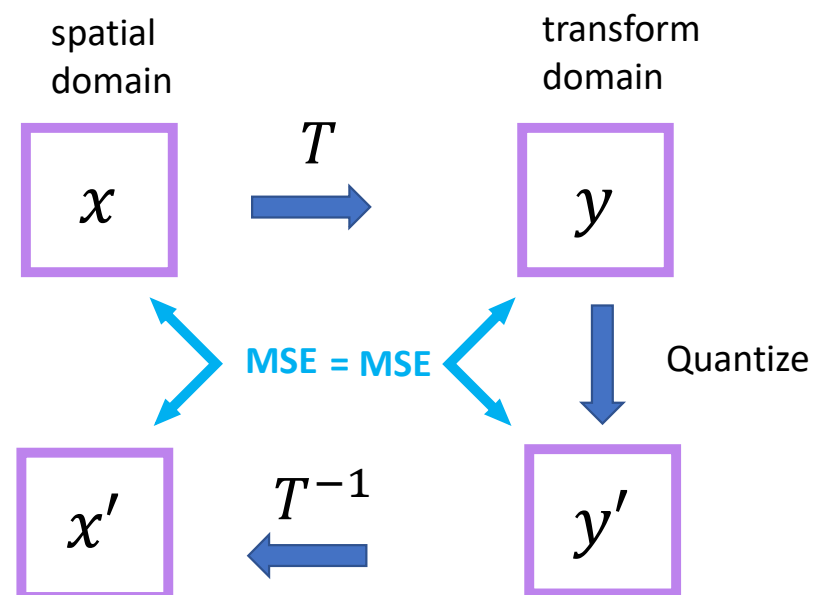
$$\left. \begin{array}{l} \text{Given: } y = Tx \\ y' = Tx' \\ T^{*t} = T^{-1} \end{array} \right\} \text{ then: } ||y - y'\| = ||x - x'\|$$

$$\text{Here, } ||x|| = \sqrt{x_1^2 + \dots + x_N^2} = \sqrt{x^{*t}x}$$

Proof: Let $a = x - x'$ and let $b = y - y'$

Then $b = Ta$, and

$$||b||^2 = b^{*t}b = (a^{*t}T^{*t})(Ta) = a^{*t}T^{-1}Ta = a^{*t}a = ||a||^2$$



DCT: Multiple definitions

- Several different definitions of the DCT exist
- For example:

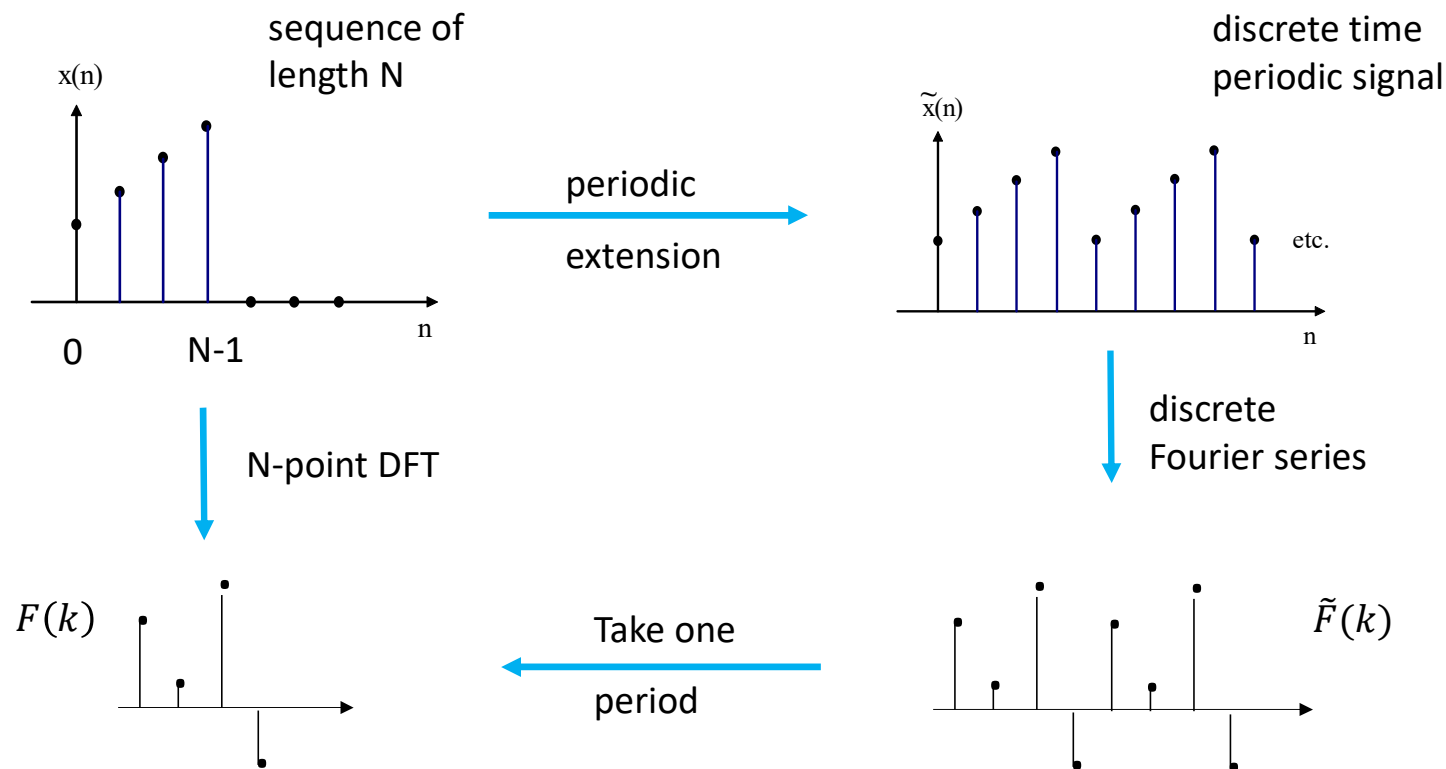
$$C(u) = \sum_{x=0}^{N-1} 2f(x) \cos \left[\frac{(2x+1)\pi u}{2N} \right]$$

$$f(x) = \frac{1}{N} \sum_{u=0}^{N-1} W(u) C(u) \cos \left[\frac{(2x+1)\pi u}{2N} \right]$$

$$W(u) = \begin{cases} \frac{1}{2} & \text{for } u = 0 \\ 1 & 1 \leq u \leq N-1 \end{cases}$$

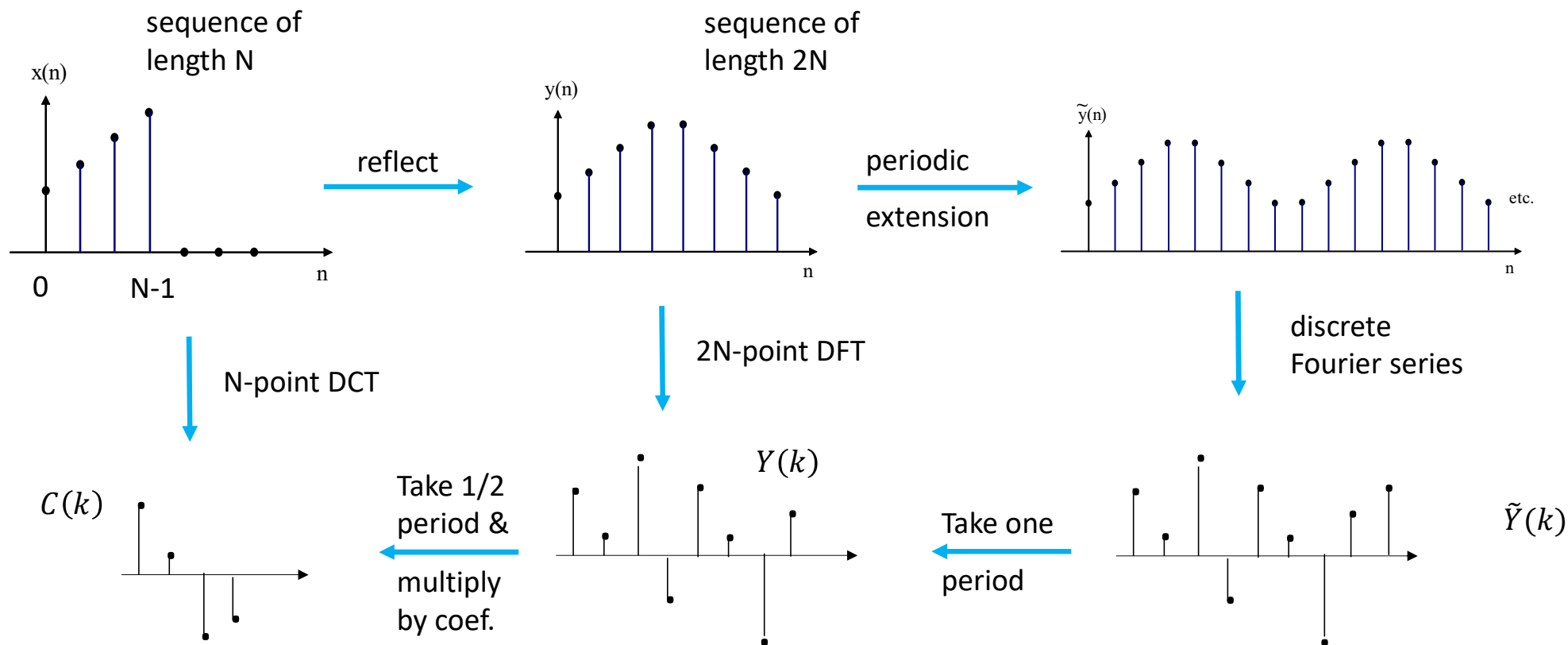
Older versions of Matlab used this definition
This version of the DCT is not unitary

DFT related to the discrete Fourier series of the periodic extension

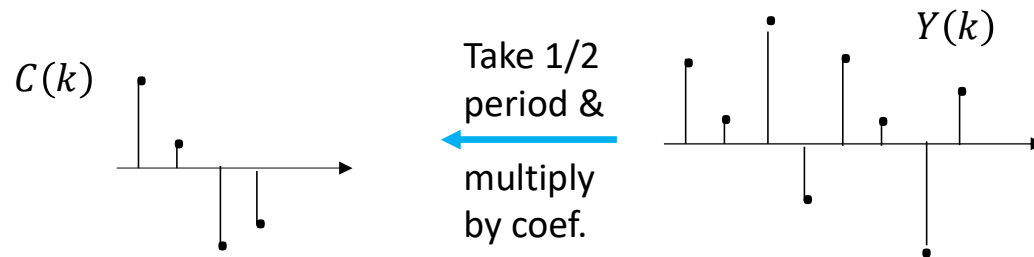


The DFT of a finite sequence $x(n)$ of length N consists of the first N coefficients of the discrete Fourier Series of the periodic extension of $x(n)$

Relationship between the DCT and the DFT



Relationship between the DCT and the DFT



$$C(k) = \sqrt{\frac{N}{2}} \alpha(k) e^{-j \frac{k\pi}{2N}} Y(k)$$

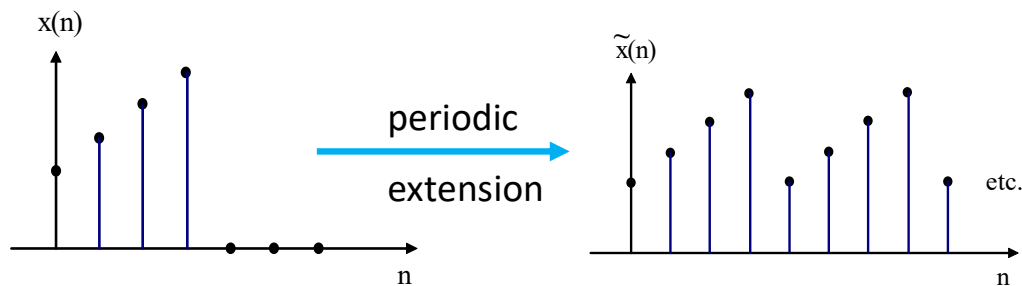
$$\alpha(u) = \sqrt{\frac{1}{N}} \text{ for } u = 0 \quad \alpha(u) = \sqrt{\frac{2}{N}} \text{ for } u = 1, 2, \dots, N-1$$

Here, $Y(k)$ is the unitary DFT:

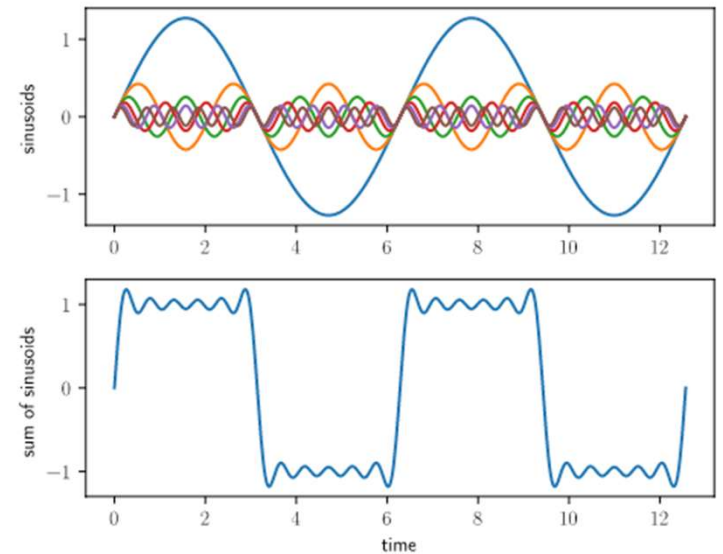
$$Y(k) = \frac{1}{\sqrt{2N}} \sum_{n=0}^{2N-1} y(n) e^{-j 2\pi n k / 2N}$$

for the 2N-point DFT

Discontinuities require high frequencies, reduce energy compaction



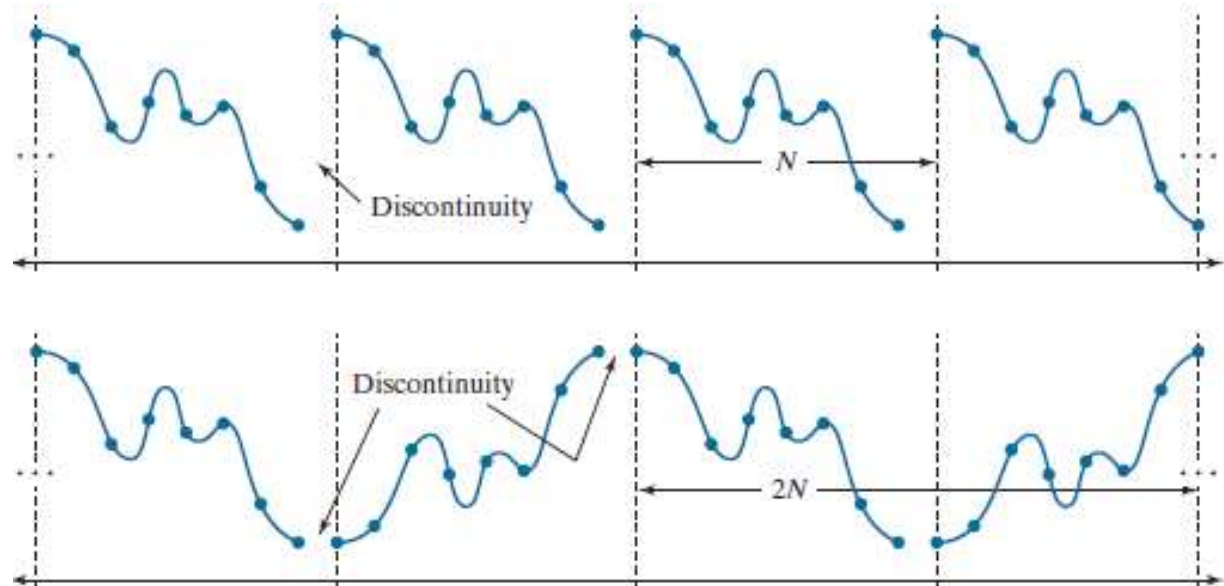
- The periodic extension has artificial discontinuities which arise from joining the start/end of $x(n)$
- These contribute to high frequencies in the DFT, reduce energy compaction



The sharp discontinuity of a square wave can be built up by adding in successively higher frequencies

Periodicity implicit in the DFT and DCT

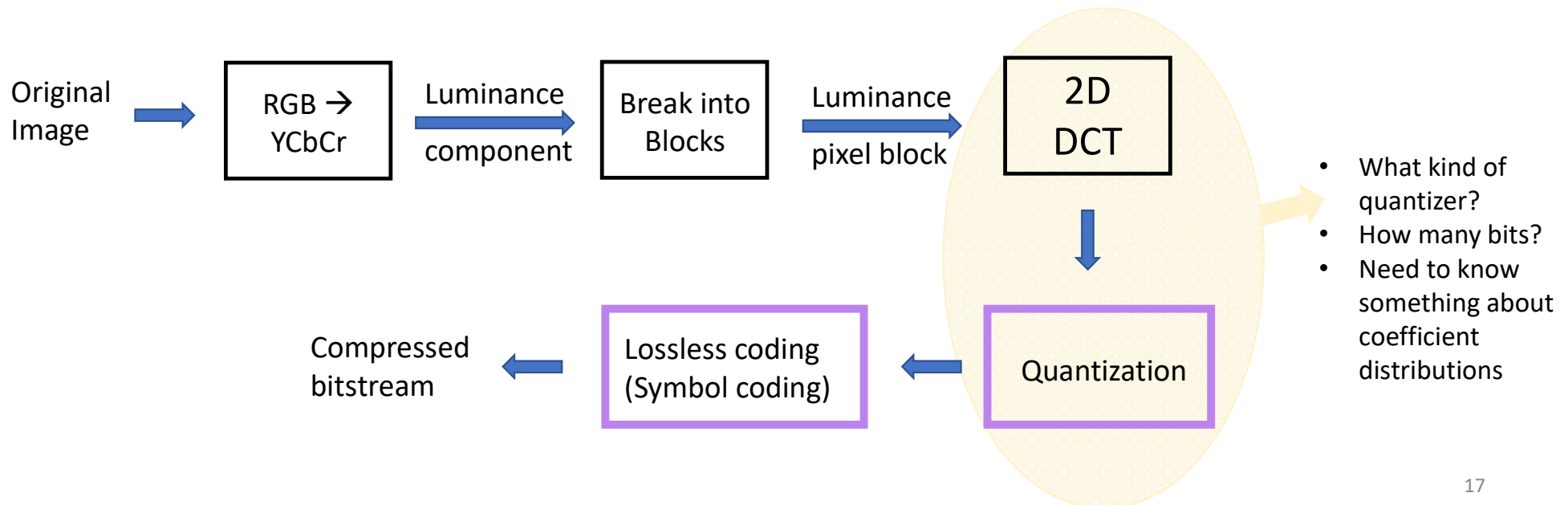
Gonzalez and Woods
Figure 6.11: Periodicity
implicit in the 1D DFT
and DCT



DCT Summary

- Linear, invertible
- Useful computationally
 - It is real (not complex like the DFT)
 - It can be computed by an FFT
 - $2N$ -point FFT required, but some symmetry
- Unitary $\rightarrow$ Distance preserving
- Several different definitions of DCT, some are not unitary
- Excellent energy compaction for highly correlated data because it doesn't have the artificial discontinuities implicit in the DFT
- After the DCT, we still have N^2 numbers to represent a block:
$$N^2 \text{ pixels} \leftrightarrow N^2 \text{ coefficients}$$
- However, the N^2 coefficients will have
 - Different distributions
 - Some which represent high frequency info
- Data compression achieved by exploiting these facts
- How to quantize the transform coefficients?

Returning to our block diagram:



DC Coefficient

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left[\frac{(2x+1)u\pi}{2N}\right] \cos\left[\frac{(2y+1)v\pi}{2N}\right]$$

$$C(0,0) = \alpha(0)\alpha(0) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \quad \alpha(0) = \sqrt{\frac{1}{N}}$$

$$C(0,0) = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y)$$

The DC coefficient $C(0,0)$ is equal to $N \times$ mean value of the block

- DC coefficient values are not negative (assuming the pixel values are not negative)
- Distribution similar to the image histogram (with the scaling factor)

AC Coefficients

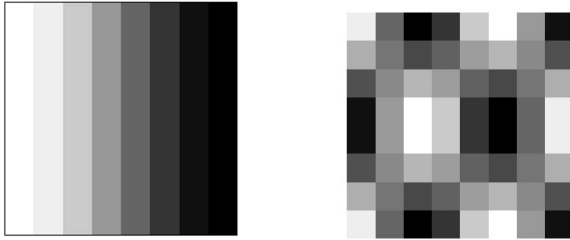
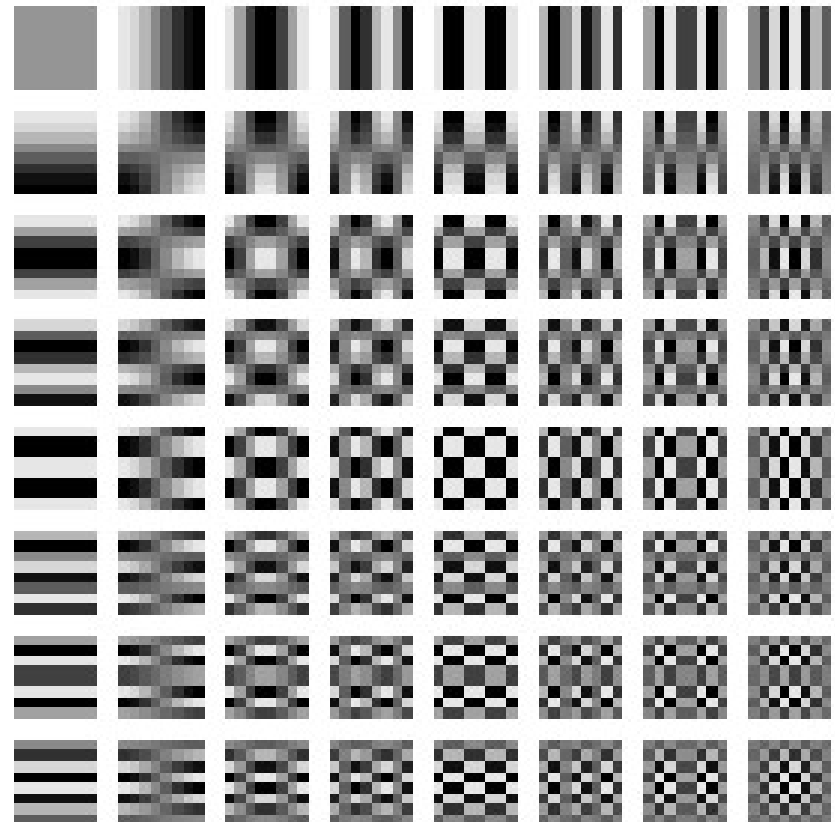
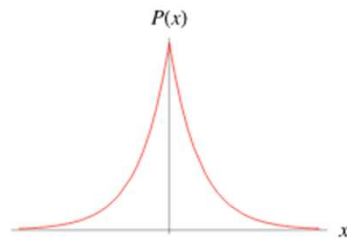


Image blocks in general would be equally likely to get brighter going left-to-right or right-to-left

AC coefficients expected to have mean zero

Roughly distributed as Laplacian with mean zero

$$f(x) = \frac{\alpha}{2} e^{-\alpha|x|}$$

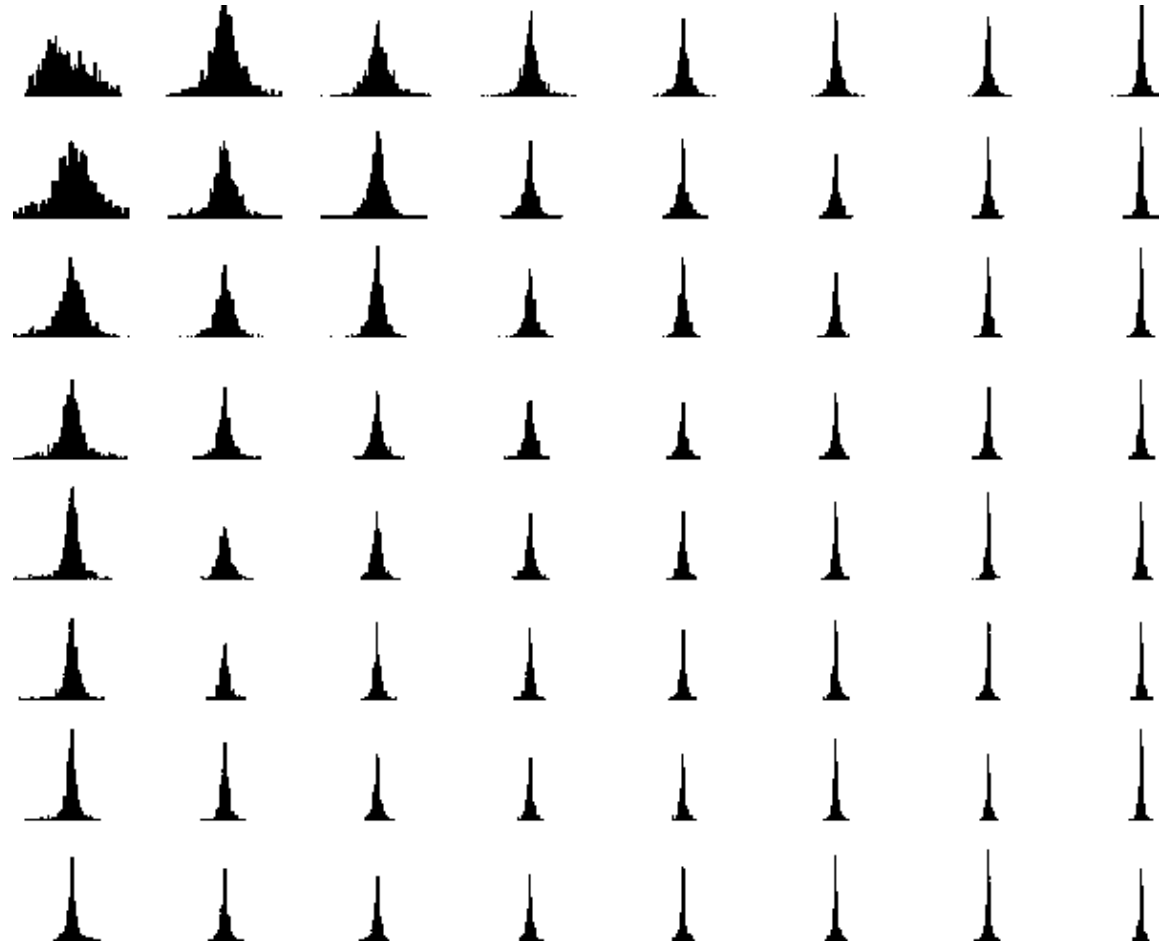


Different variances for the different coefficient positions

Distributions of DCT coefficients

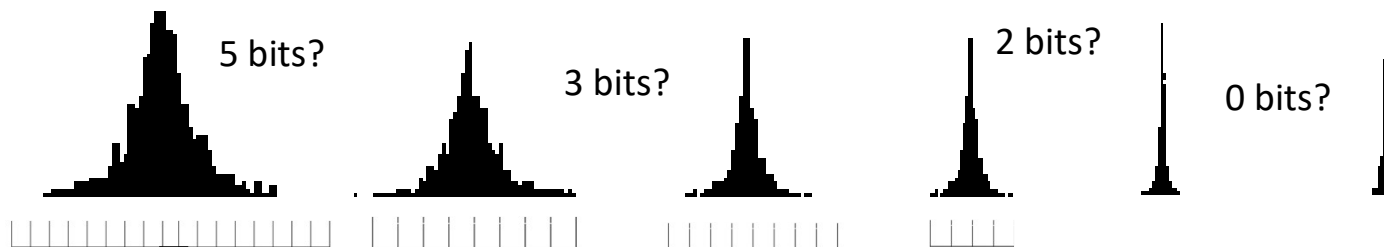


Bridge Image, with
distributions of DCT
coefficients arising
from 8x8 blocks



Quantizing after transforming

- For each of the 64 positions, we could assign a quantizer with some number of levels
- Need to consider the variance of each position



If we decide to quantize some coefficient position using 0 bits, what mean squared error is incurred?

$$\mathbb{E} = E[(u - \hat{u})^2] = E[(u - \mu)^2] = E[u^2] = \sigma^2$$

Zonal Bit Allocation

- Coefficient positions with tiny variance, can give 0 bits.
 - Decoder will assume coefficient equals mean value (zero) for that position
- For position with slightly larger variance, transmit 1 bit. Reconstruct at $+1/\alpha$ or $-1/\alpha$
- Overall, for each position, might choose a different number of bits to be sent
- Display those numbers as a matrix, called a *zonal bit allocation matrix*

8	7	6	4	3	2	1	0
7	6	5	4	3	2	1	0
6	5	4	3	3	1	1	0
4	4	3	3	2	1	0	0
3	3	3	2	1	1	0	0
2	2	1	1	1	0	0	0
1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0

Sum up values in the array:
123 bits for 64 coefficients
→ 123 bits for 64 pixels
→ 1.92 bpp

How to choose good values?

The Bit Allocation Problem

- Suppose we have k random variables $X_1, X_2, \dots, X_k$ and the pdfs are known
- Variance is σ_i^2 for X_i
- Can design optimal MSE quantizer Q_i for X_i given b_i bits
- Let $W_i(b_i)$ denote the MSE incurred in quantizing X_i with b_i bits of resolution

Find $b_i, i = 1, 2, \dots, k$ to minimize

$$\sum_{i=1}^k W_i(b_i)$$

such that $\sum_{i=1}^k b_i \leq B$

Here, B is the total number of bits available

The Bit Allocation Problem

Log-Variance Bit Allocation

Suppose we have 2 random variables X_1 and X_2

$$X_1 \sim \text{unif} [-a_1, +a_1]$$

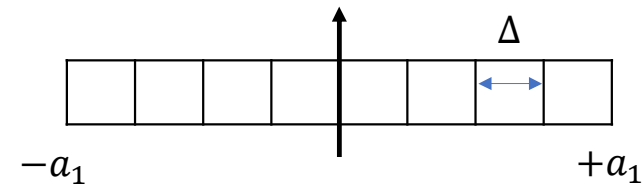
$$X_2 \sim \text{unif} [-a_2, +a_2]$$

The variances are:

$$\sigma_1^2 = E[(X_1 - \mu_x)^2] = E(X_1^2) = \int_{-a_1}^{+a_1} \frac{x^2}{2a_1} dx = \frac{a_1^2}{3}$$

$$\sigma_2^2 = \frac{a_2^2}{3}$$

Suppose we quantize X_1 with b_1 bits, spaced uniformly. Then we would have 2^{b_1} levels.



The spacing between levels is

$$\Delta = \frac{2a_1}{2^{b_1}}$$

The expected value of the squared error is:

$$\int_{-a_1}^{+a_1} (x - r_x)^2 \frac{1}{2a_1} dx = \frac{2^{b_1}}{2a_1} \int_{-\Delta/2}^{+\Delta/2} x^2 dx =$$

r_x denotes the quantization level for input x

$$\int_{-\Delta/2}^{+\Delta/2} x^2 \frac{1}{\Delta} dx = \frac{\Delta^2}{12} = \sigma_1^2 2^{-2b_1}$$

Log-Variance Bit Allocation

Assumption: We want to have the same expected error for each random variable

$$\sigma_1^2 2^{-2b_1} = \sigma_2^2 2^{-2b_2} \quad \leftarrow \text{MSE the same for the 2 variables}$$

$$b_1 + b_2 = B \quad \leftarrow \text{Constraint on the total bit budget}$$

Take $\log_2$

$$\log_2 \sigma_1^2 - 2b_1 = \log_2 \sigma_2^2 - 2B + 2b_1$$

Define: $\bar{b} = \frac{B}{2}$ = average #bits per component

Define: $\rho^2 = (\sigma_1^2 \sigma_2^2)^{1/2} = \sigma_1 \sigma_2$
= geometric mean of the variances of the random variables

Log-Variance Bit Allocation

$$b_1 = \frac{1}{2} \log_2 \sigma_1^2 + \bar{b} - \frac{1}{2} \log_2 \rho^2$$

$$b_1 = \bar{b} + \frac{1}{2} \log_2 \frac{\sigma_1^2}{\rho^2}$$



if $\sigma_1 > \rho$ then $b_1 > \bar{b}$

if $\sigma_1 < \rho$ then $b_1 < \bar{b}$

For k random variables,

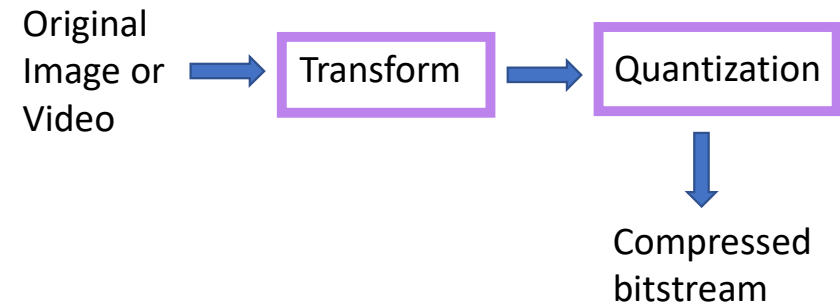
Define: $\bar{b} = \frac{B}{k}$ = average #bits per component

Define: $\rho^2 = \left(\prod_{i=1}^k \sigma_i^2 \right)^{1/k}$

= geometric mean of the variances of the random variables

$$b_i = \bar{b} + \frac{1}{2} \log_2 \frac{\sigma_i^2}{\rho^2}$$

called log-variance bit allocation (need to impose non-negativity)



Zonal Bit Allocation

8	7	6	4	3	2	1	0
7	6	5	4	3	2	1	0
6	5	4	3	3	1	1	0
4	4	3	3	2	1	0	0
3	3	3	2	1	1	0	0
2	2	1	1	1	0	0	0
1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0

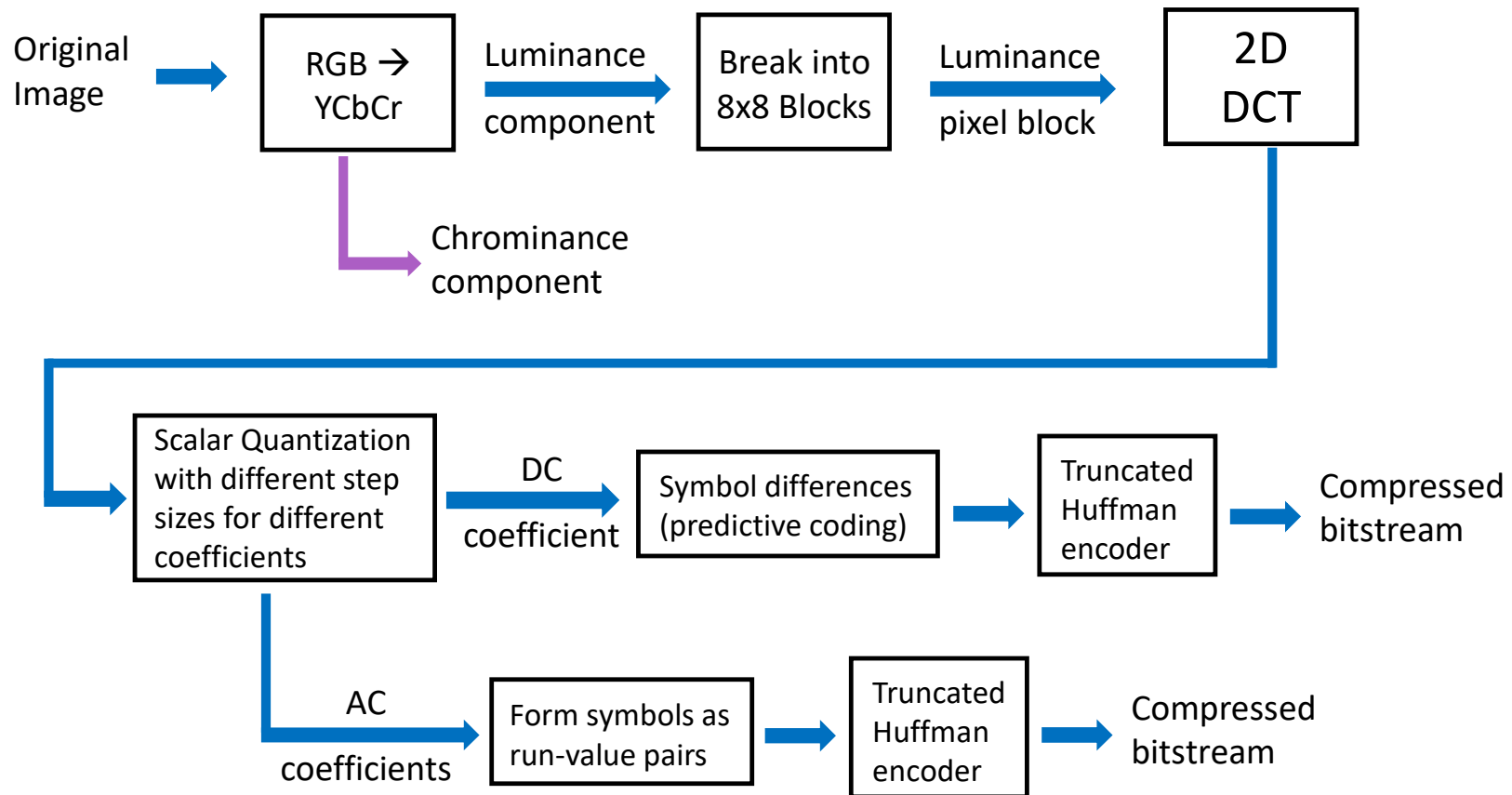
Advantages:

Simplicity + if no Huffman coding, then output rate is known, and no error propagation

Disadvantages:

- Always using the same number of bits for a coefficient in a given position
 - If the coefficient value is close to the mean, then that should have been predictable
 - Don't want to use so many bits in that case
- Just use lots of bits for cases where the coefficients depart significantly from their mean
- *Threshold coding*, used in JPEG

JPEG Encoder Block Diagram:



Run Length Coding

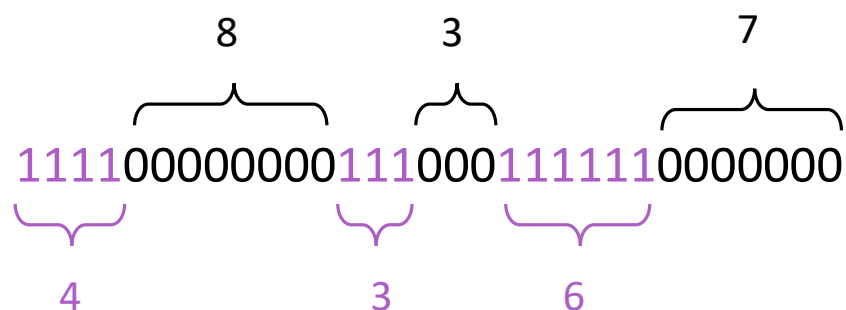
- Suppose your input strings are binary with runs of zeroes and ones:

1111000000001110001111110000000

- Given: $p(0) = 0.6$, $p(1) = 0.4$
- Can encode this with, say:
 - arithmetic encoder with probs 0.6 and 0.4
 - Huffman encoder on blocks

Suppose the input symbols are not i.i.d.

- 1's tend to occur next to other 1's
- 0's tend to occur next to other 0's



Alternatively, can encode the run lengths:

4 8 3 3 6 7

Use a Huffman coder for these run lengths

Run Length Coding

Two issues:

1. Need to specify what the first symbol is
2. If the Huffman code is designed for all run lengths from 1 to N, what happens if we encounter a run $> N$?

Several ways to solve these problems

- Explicitly encode the first symbol (e.g., with a fixed length code)
- Assume that first pixel is white, but allow run-lengths to equal zero
 - If first pixel is black, then send codeword for zero length run
- This would solve the second problem too

Design Huffman code for run lengths from 0 to 40

→ encounter a run of length 95

→ send codewords for 40, 0, 40, 0, 15

Run Value Coding

- If the input has runs of zeroes interspersed with non-zero values:
- Example: 000200050100000013000004
- Can make (run length, value) pairs:



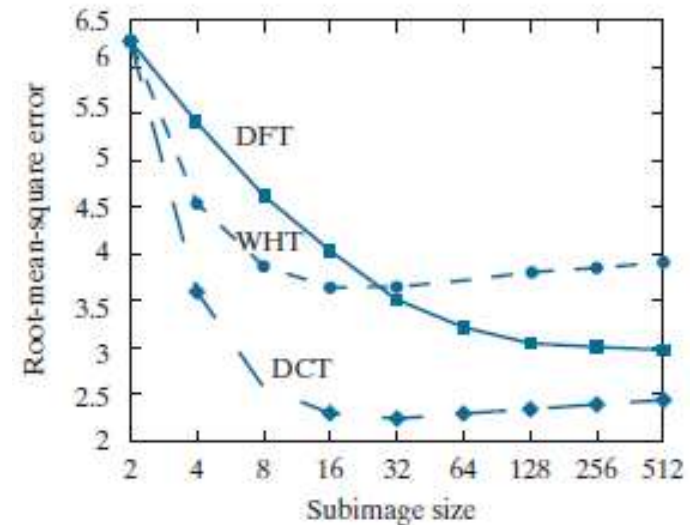
- Can design a Huffman code for these pairs
- This is called *run value coding*

JPEG Block Size

- JPEG uses blocks of size 8x8 pixels
 - Larger blocks: require more computational complexity
 - Smaller blocks: wouldn't capture much of the spatial correlation present between pixels
- 8x8 was seen as a good tradeoff

This tradeoff depends on the underlying resolution

- Higher resolution images today
- More compute power today



Gonzalez and Woods Fig 8.23: Reconstruction error from taking blocks of different sizes, zeroing out 75% of the coefficients (the higher frequency ones), and taking the inverse transform

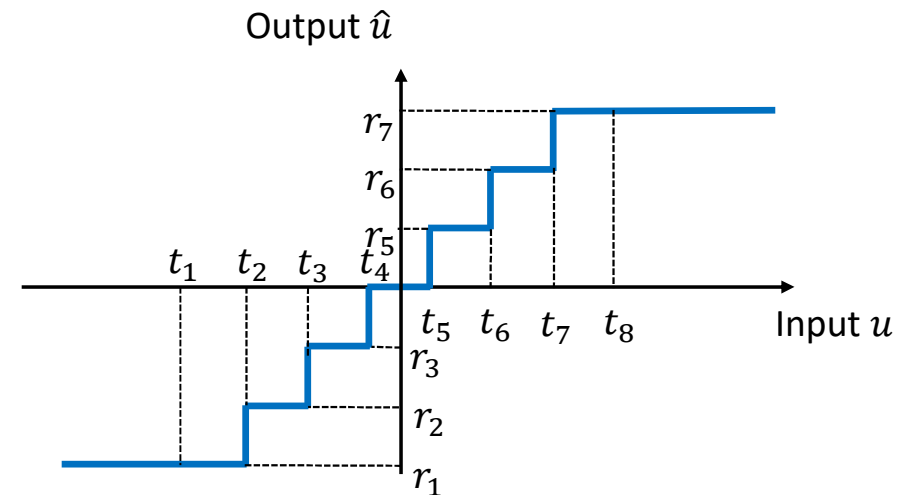
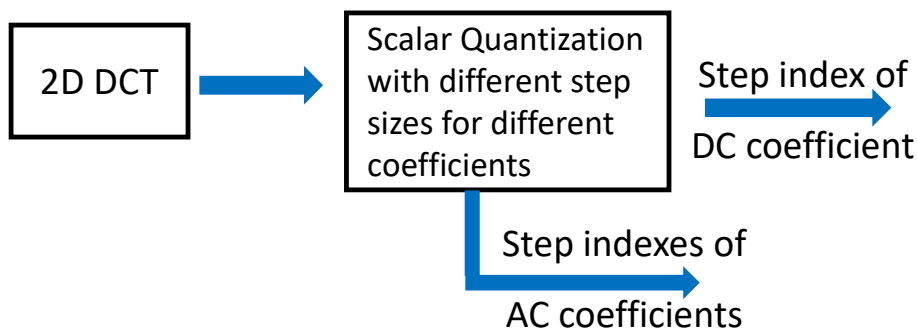
Reminder: Quantization can be separated into Encoder & Decoder

We think of this quantizer as having two separate operations:

Encoder $\mathcal{E}: R \rightarrow I = \{1, 2, \dots, L\}$
which is a set of step indexes

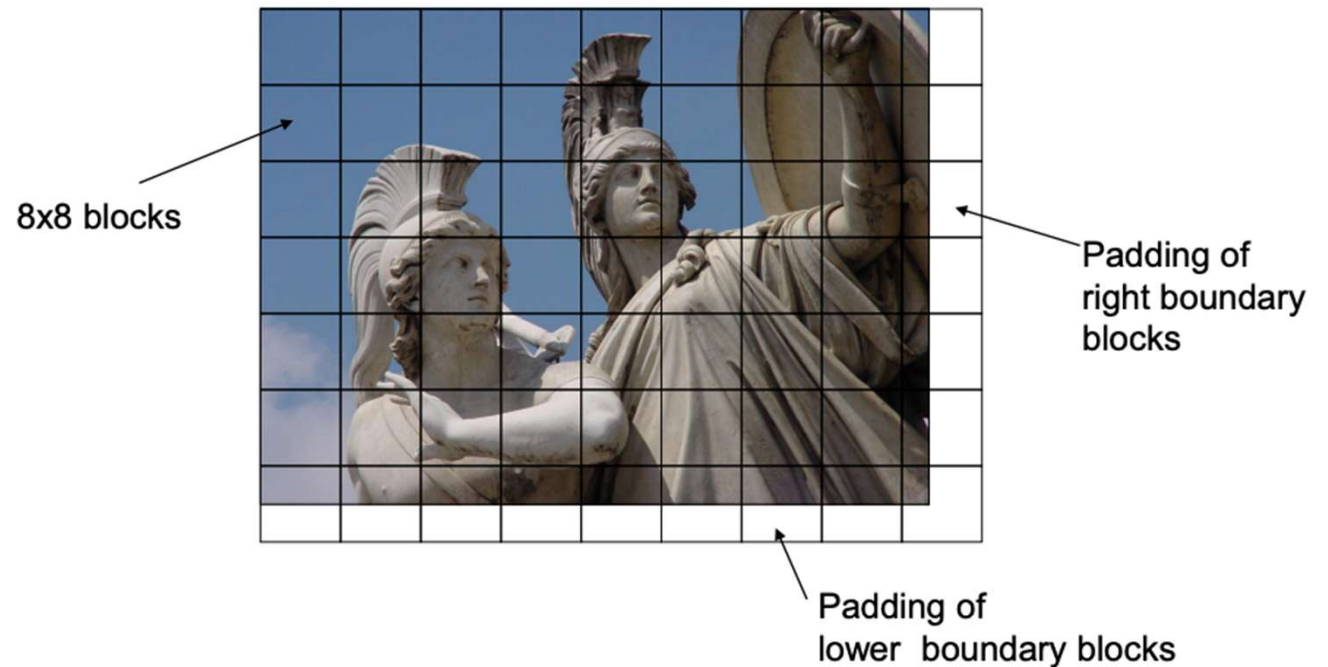
Decoder $\mathcal{D}: I \rightarrow \mathcal{C} = \{r_1, r_2, \dots, r_L\}$
which maps the step index to the value of the step

In the JPEG Encoder block diagram, the Quantizer means the *Quantizer Encoder*, so it outputs step indexes:



JPEG Preliminary Steps

- RGB $\rightarrow$ YCbCr (or some other color transform)
- Break into 8x8 blocks
- Might need to pad if the image size is not a multiple of 8

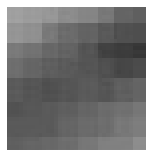


From: Bernd Girod, Stanford

JPEG Preliminary Steps

- Starting with original (luminance) pixel block, shift by -128
- Range now goes from -128 to +127

Since the cosine function is symmetric about zero, making the data also symmetric about zero reduces internal precision requirements for computing the DCT



160	155	151	140	133	120	100	95
149	144	121	107	99	89	80	78
122	114	103	99	84	65	60	55
110	101	94	90	85	80	70	75
90	85	80	85	90	85	95	100
82	87	90	95	101	108	112	119
89	90	90	101	106	109	120	125
95	100	104	109	119	128	129	141

32	27	23	12	5	-8	-28	-33
21	16	-7	-21	-29	-39	-48	-50
-6	-14	-25	-29	-44	-63	-68	-73
-18	-27	-34	-38	-43	-48	-58	-53
-38	-43	-48	-43	-38	-43	-33	-28
-46	-41	-38	-33	-27	-20	-16	-9
-39	-38	-38	-27	-22	-19	-8	-3
-33	-28	-24	-19	-9	0	1	13

JPEG DCT2 Step

- Next we perform a 2-d forward DCT (shown here as integers)
- Values not all equally important perceptually
- We will next quantize these values
- JPEG uses different step sizes for different coefficients
 - Large step size: good compression, poor reproduction quality
 - Small step size: good reproduction quality, poor compression
- Consider frequency characteristics of the Human Visual System → want to quantize high frequencies coarsely

The diagram shows an 8x8 grid of DCT coefficients. The top-left cell (0,0) is highlighted in yellow and labeled 'DC coefficient' with a blue arrow. The top-right cells (0,1) through (0,7) are highlighted in light blue and labeled 'low frequency coefficients' with a blue arrow. The bottom-left cells (7,0) through (7,7) are highlighted in light blue and labeled 'middle frequency coefficients' with a blue arrow. The bottom-right cells (6,6) through (7,7) are highlighted in light orange and labeled 'high frequency coefficients' with a blue arrow.

-201	40	9	3	9	-5	-1	-1
19	129	-6	7	0	0	1	1
106	0	-7	1	-5	-3	3	-1
34	-29	-3	0	-2	0	0	1
25	-8	-5	2	1	-2	6	1
2	6	-10	0	2	3	2	-1
5	-1	-12	-3	-1	1	3	0
-12	1	-2	-2	5	4	-6	3

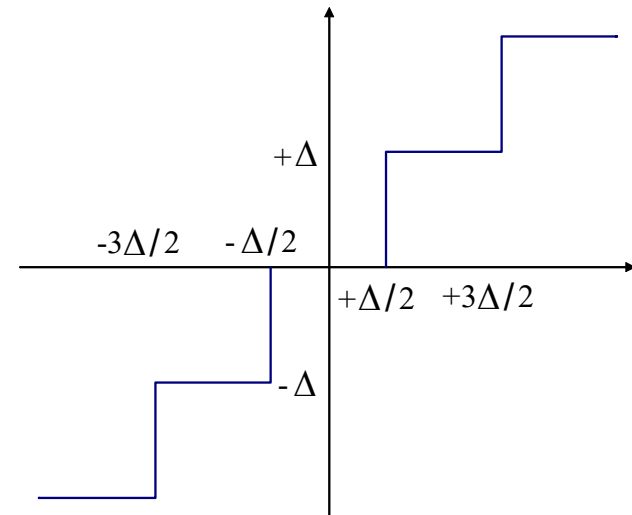
JPEG Quantization

- Suppose we quantize the DC coefficient with a step size $\Delta=16$
- The step index for the coefficient is

$$i = \text{round}\left(\frac{x}{\Delta}\right) = \text{round}\left(\frac{-201}{16}\right) = -13$$

- The decoder will reconstruct it as

$$y = \Delta \times i = 16 \times (-13) = -208$$



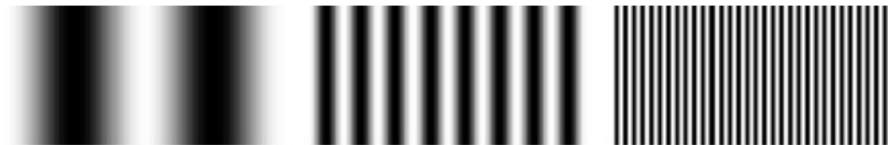
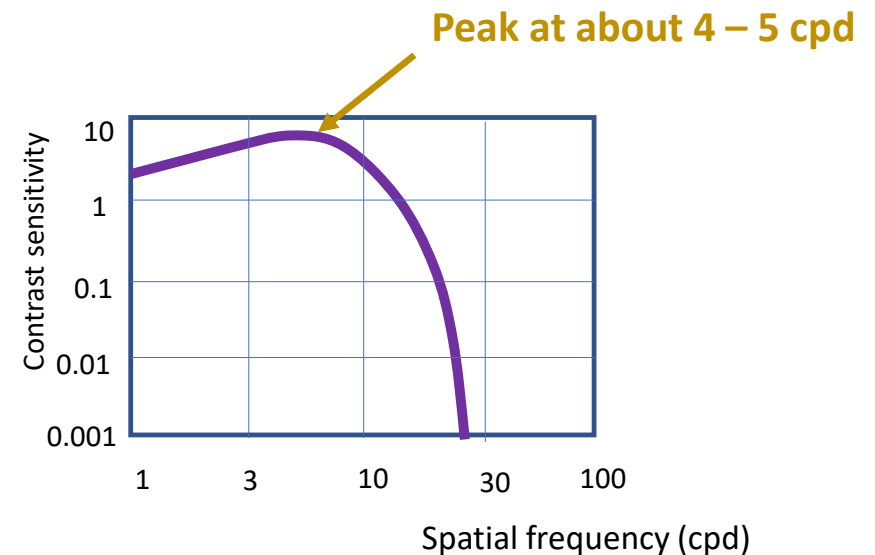
JPEG Default Quantization Table (Luminance)

Table of quantization step sizes = Q Table

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

- Table was determined by measuring experimentally the sensitivity of observers to different frequencies
- Table is not symmetric

- Table is not quite monotonic

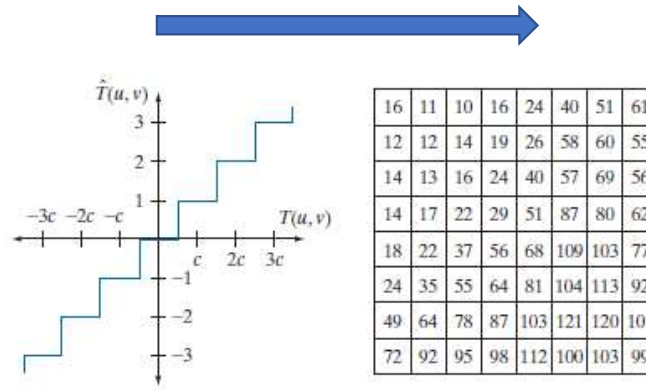


Coefficients are Quantized to obtain Array of Step Indexes

-201	40	9	3	9	-5	-1	-1
19	129	-6	7	0	0	1	1
106	0	-7	1	-5	-3	3	-1
34	-29	-3	0	-2	0	0	1
25	-8	-5	2	1	-2	6	1
2	6	-10	0	2	3	2	-1
5	-1	-12	-3	-1	1	3	0
-12	1	-2	-2	5	4	-6	3

DCT Coefficients (displayed as integers)

Each coefficient is quantized with the quantizer step index specified in the table



16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Q Table

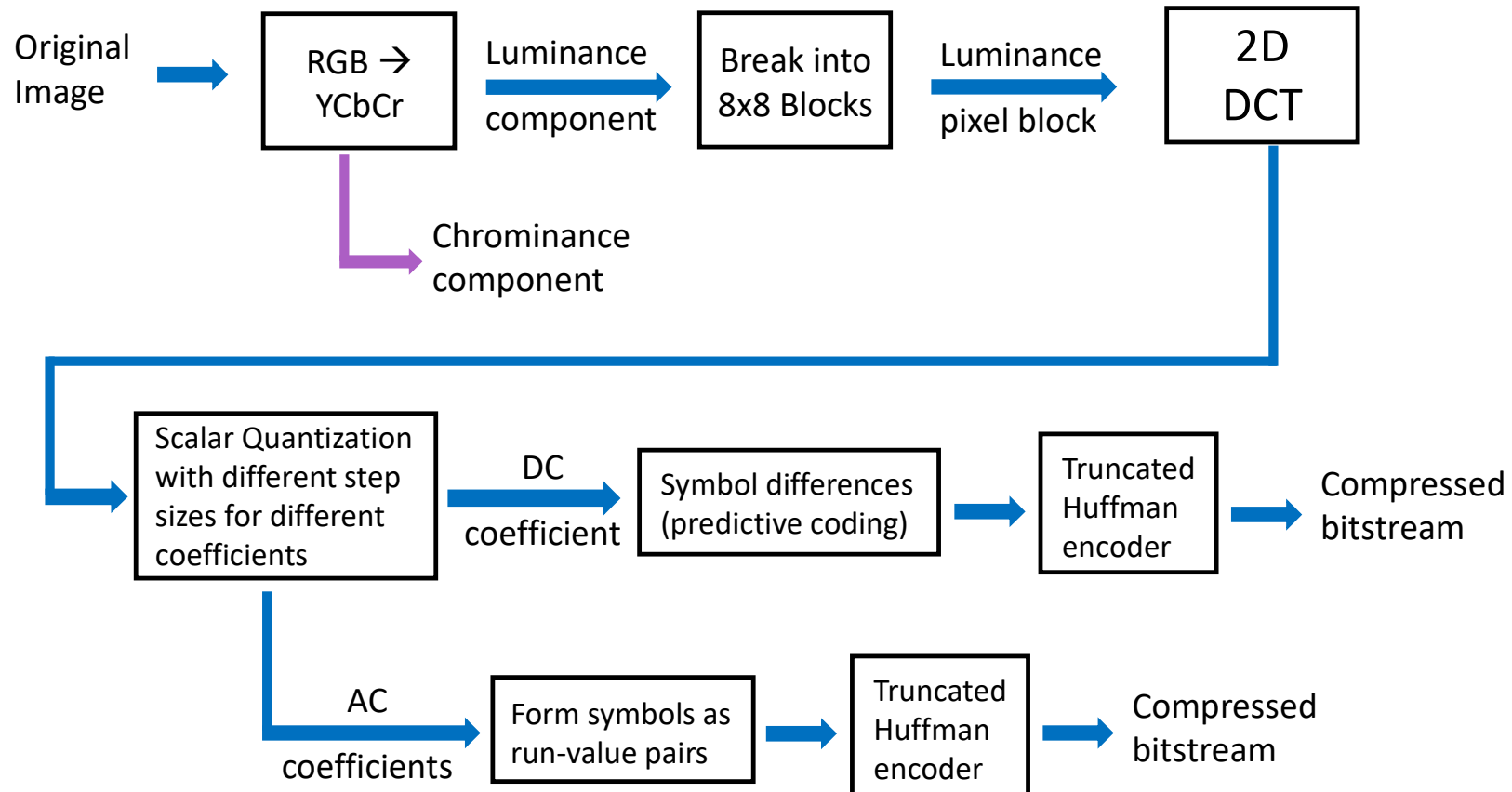
-13	4	1	0	0	0	0	0
2	11	0	0	0	0	0	0
8	0	0	0	0	0	0	0
2	-2	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Array of step indexes

These step indices will now be losslessly encoded with variable length coding

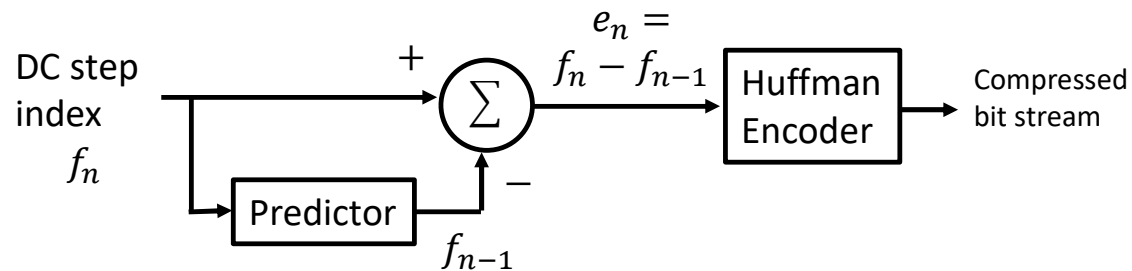
- No further degradation

JPEG Encoder Block Diagram



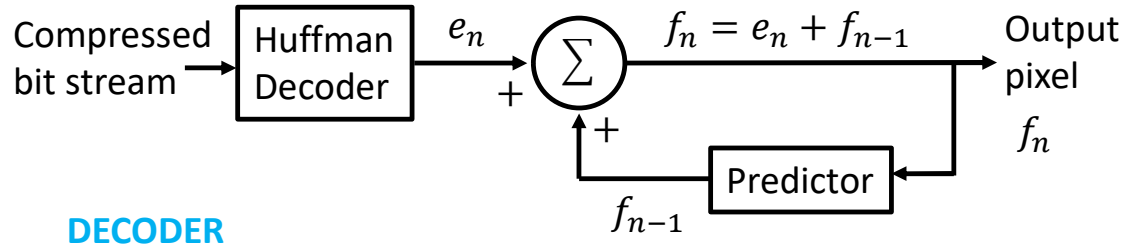
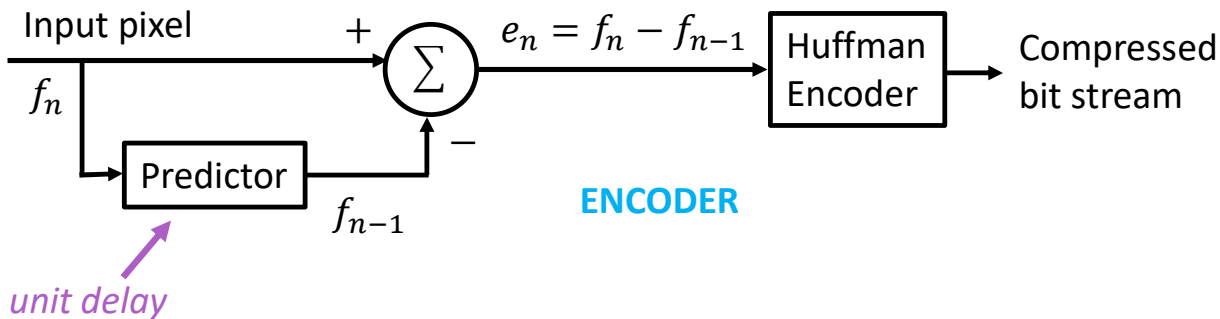
JPEG: DC Coefficient Encoding

- High correlation between DC coefficients of adjacent blocks
- So the DC step index is encoded as the difference with the DC step index of the previous block
- A truncated Huffman code is used to encode this difference



Lossless predictive coding

- We can use past pixels to make a prediction of the next pixel
- Simple case: Each pixel is predicted to be the same as the previous one



Coding example: input pixels

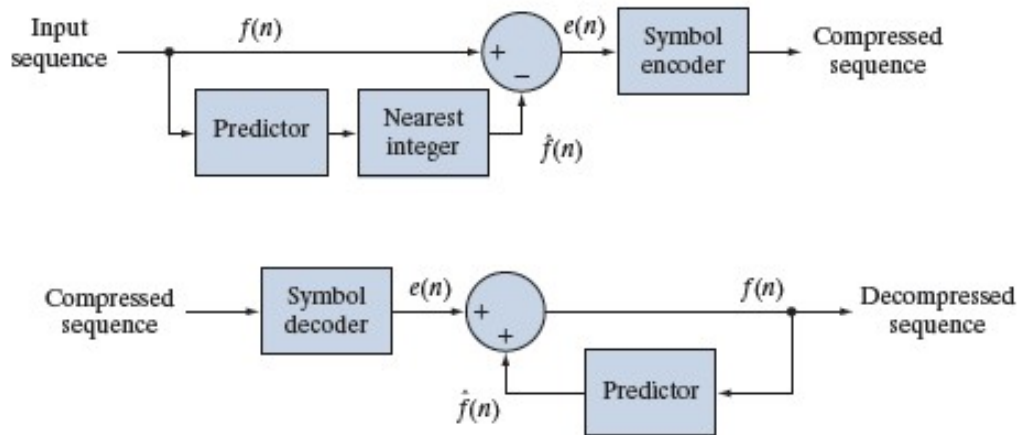
128	132	138	111	112	112	101	100	99
-----	-----	-----	-----	-----	-----	-----	-----	----

Prediction error actually coded:

128	4	6	-27	1	0	-11	-1	-1
-----	---	---	-----	---	---	-----	----	----

Lossless predictive coding

Gonzalez & Woods, Figure 8.30



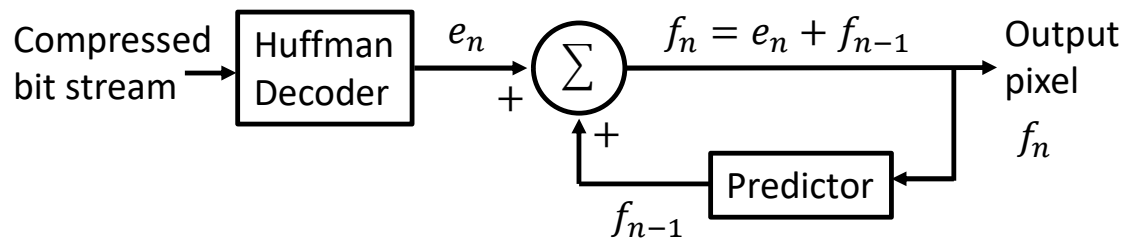
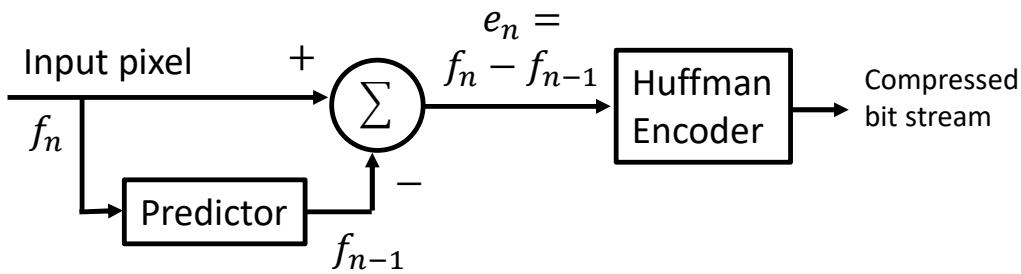
Prediction of next pixel can be more complicated than just using the 1 previous pixel

$$\hat{f}(x, y) = \text{round} \left[\sum_{i=1}^m \alpha_i f(x, y - i) \right]$$

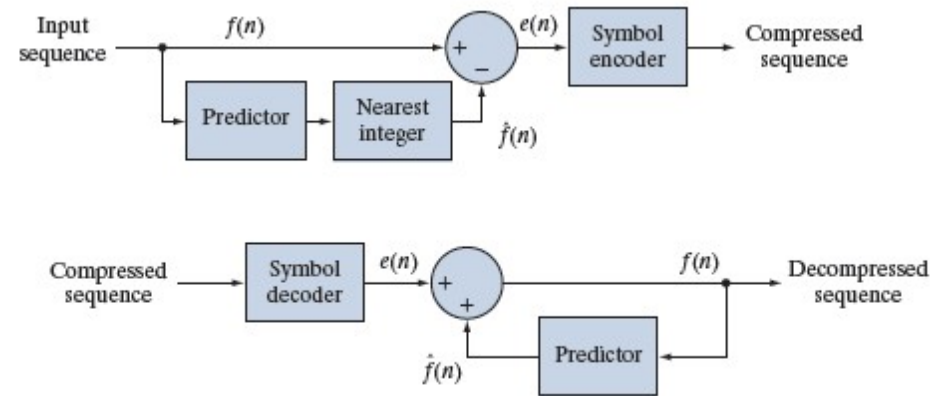
Prediction is not restricted to pixels in the same row, for example:

$$\hat{f}(x, y) = \text{round} \left[\frac{f(x, y - 1) + f(x - 1, y)}{2} \right]$$

Lossless predictive coding



Gonzalez & Woods, Figure 8.30



$$\text{Encoder: } e(n) = f(n) - \hat{f}(n)$$

$$\text{Decoder: } f(n) = e(n) + \hat{f}(n)$$

Reminder: Modified / Truncated Huffman Coding

Symbol	Prob.
A	.25
B	.23
C	.18
D	.11
E	.10
F	.07
Group	.06

Symbol	Prob.	Huffman Design Output
A	.25	00
B	.23	10
C	.18	010
D	.11	110
E	.10	111
F	.07	0110
Group	.06	0111

complete
codewords
for symbols

Base code

If you need to transmit one of the group, need to transmit the base code + fixed length 3-bit word to say which one of the group it is

Symbol	Prob.	Codeword
A	.25	00
B	.23	10
C	.18	010
D	.11	110
E	.10	111
F	.07	0110
G	.01	0111000
H	.01	0111001
I	.009	0111010
J	.009	0111011
K	.008	0111100
L	.008	0111101
M	.003	0111110
N	.003	0111111

JPEG: Huffman Tables for DC Coefficient Encoding

JPEG uses lots of groups, of different sizes

Our DC coefficient has value -13

Suppose the previous block had value -16

$$-13 - (-16) = +3$$

1) Figure out which group you're in

Range	DC Difference Category
0	0
-1, 1	1
-3, -2, 2, 3	2
-7, ..., -4, 4, ..., 7	3
-15, ..., -8, 8, ..., 15	4
-31, ..., -16, 16, ..., 31	5
-63, ..., -32, 32, ..., 63	6
-127, ..., -64, 64, ..., 127	7
-255, ..., -128, 128, ..., 255	8
-511, ..., -256, 256, ..., 511	9
-1023, ..., -512, 512, ..., 1023	A
-2047, ..., -1024, 1024, ..., 2047	B
-4095, ..., -2048, 2048, ..., 4095	C
-8191, ..., -4096, 4096, ..., 8191	D
-16383, ..., -8192, 8192, ..., 16383	E
-32767, ..., -16384, 16384, ..., 32767	F

+3 is in category 2

2) Look up the base code for that group

Category	Base Code	Length	Category	Base Code	Length
0	010	3	6	1110	10
1	011	4	7	11110	12
2	100	5	8	111110	14
3	00	5	9	1111110	16
4	101	7	A	11111110	18
5	110	8	B	111111110	20

The base code for category 2 is 100

3) Determine the fixed length part

Category 2 Elements	-3	-2	+2	+3
Fixed length part	00	01	10	11

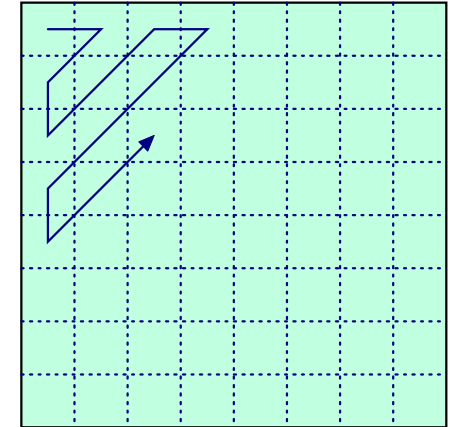
The complete codeword for +3 is 10011

JPEG: AC Coefficient Encoding uses Run-Value Coding

- Coarse quantization of high frequency coefficients has produced many zeroes

-13	4	1	0	0	0	0	0
2	11	0	0	0	0	0	0
8	0	0	0	0	0	0	0
2	-2	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

- Want to encode these efficiently
- Arrange coefficients so high probability of having many zeroes in a row: *zig zag scan*



- In zigzag order (ignoring DC coefficient), we make symbols using:

(run length of 0's, next non-zero coefficient)

The symbols come out:

(0,4),(0,2),(0,8),(0,11),(0,1)(3,2),(0,1),(0,-2),EOB

EOB denotes *end-of-block* (Meaning all coefficients after that point are zero in zigzag order)

JPEG: AC Coefficient Encoding

Range	DC Difference Category	AC Category
0	0	N/A
-1, 1	1	1
-3, -2, 2, 3	2	2
-7, ..., -4, 4, ..., 7	3	3
-15, ..., -8, 8, ..., 15	4	4
-31, ..., -16, 16, ..., 31	5	5
-63, ..., -32, 32, ..., 63	6	6
-127, ..., -64, 64, ..., 127	7	7
-255, ..., -128, 128, ..., 255	8	8
-511, ..., -256, 256, ..., 511	9	9
-1023, ..., -512, 512, ..., 1023	A	A
-2047, ..., -1024, 1024, ..., 2047	B	B
-4095, ..., -2048, 2048, ..., 4095	C	C
-8191, ..., -4096, 4096, ..., 8191	D	D
-16383, ..., -8192, 8192, ..., 16383	E	E
-32767, ..., -16384, 16384, ..., 32767	F	N/A

Let's encode one of the AC run-value pairs: (0,8)

Same 3 steps, but now using the Huffman tables for AC coefficient coding

1) Figure out which group you're in

+8 is in AC category 4

JPEG: AC Coefficient Encoding

2) Look up the base code for run length = 0, category = 4

Run/Category	Base Code	Length	Run/Category	Base Code	Length
0/0	1010 (= EOB)	4	8/1	11111010	9
0/1	00	3	8/2	11111111000000	17
0/2	01	4	8/3	111111110110111	19
0/3	100	6	8/4	111111110111000	20
0/4	1011	8	8/5	111111110111001	21
0/5	11010	10	8/6	111111110111010	22
0/6	111000	12	8/7	111111110111011	23
0/7	1111000	14	8/8	111111110111100	24
0/8	111110110	18	8/9	111111110111101	25
0/9	111111110000010	25	8/A	111111110111110	26
0/A	111111110000011	26	9/1	111111000	10
1/1	1100	5	9/2	111111110111111	18
1/2	111001	8	9/3	111111111000000	19
1/3	1111001	10	9/4	111111111000001	20
1/4	111110110	13	9/5	111111111000010	21
1/5	11111110110	16	9/6	111111111000011	22
1/6	111111110000100	22	9/7	111111111000100	23
1/7	111111110000101	23	9/8	111111111000101	24
1/8	111111110000110	24	9/9	111111111000110	25
1/9	111111110000111	25	9/A	111111111000111	26
1/A	111111110001000	26	A/1	111111001	10
2/1	11011	6	A/2	111111111001000	18
2/2	11111000	10	A/3	111111111001001	19
2/3	1111110111	13	A/4	111111111001010	20
2/4	111111110001001	20	A/5	111111111001011	21
2/5	111111110001010	21	A/6	111111111001100	22
2/6	111111110001011	22	A/7	111111111001101	23
2/7	111111110001100	23			

2/8	111111110001101	24	A/8	111111111001110	24
2/9	111111110001110	25	A/9	111111111001111	25
2/A	111111110001111	26	A/A	111111111010000	26
3/1	111010	7	B/1	111111010	10
3/2	111110111	11	B/2	111111111010001	18
3/3	1111110111	14	B/3	111111111010010	19
3/4	111111110010000	20	B/4	111111111010011	20
3/5	111111110010001	21	B/5	111111111010100	21
3/6	111111110010010	22	B/6	111111111010101	22
3/7	111111110010011	23	B/7	111111111010110	23
3/8	111111110010100	24	B/8	111111111010111	24
3/9	111111110010101	25	B/9	111111111011000	25
3/A	111111110010110	26	B/A	111111111011001	26
4/1	111011	7	C/1	1111111010	11
4/2	1111111000	12	C/2	111111111011010	18
4/3	111111110010111	19	C/3	111111111011011	19
4/4	111111110011000	20	C/4	111111111011100	20
4/5	111111110011001	21	C/5	111111111011101	21
4/6	111111110011010	22	C/6	111111111011110	22
4/7	111111110011011	23	C/7	111111111011111	23
4/8	111111110011100	24	C/8	111111111100000	24
4/9	111111110011101	25	C/9	111111111100001	25
4/A	111111110011110	26	C/A	111111111100010	26

3) Determine the fixed length part

+8 corresponds to 1000 → full codeword is 10111000

What about the chrominance?

Chrominance planes Cb & Cr are:

- spatially filtered to reduce aliasing
- downsampled by a factor of 2 in each direction
- then DCT2 / quantization / Huffman like the luminance but with more severe quantization

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

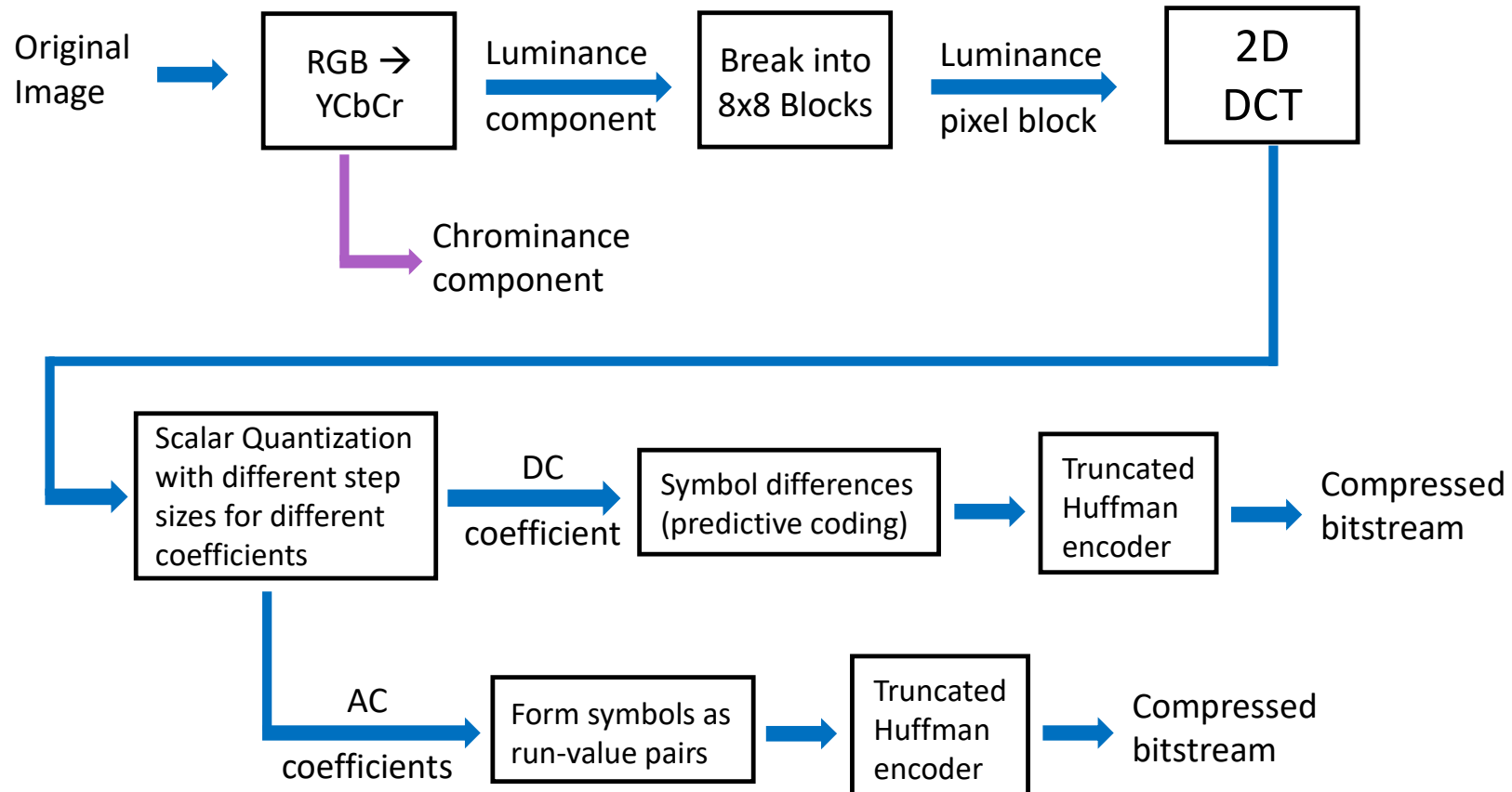
Luminance Q Table

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

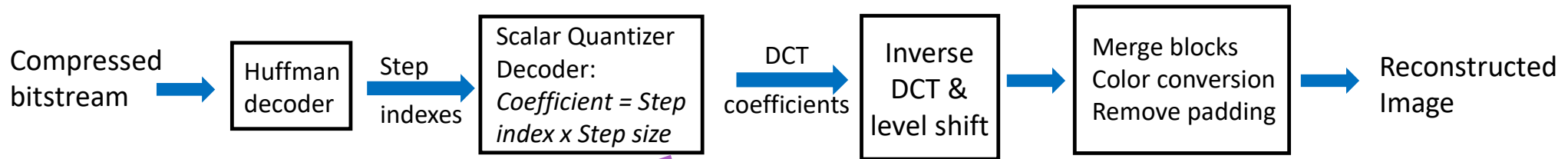
Chrominance Q Table

People have little difficulty
resolving shapes when color
info bleeds slightly across
the grayscale edges

JPEG Encoder Block Diagram



JPEG Decoder block diagram



-208	44	10	0	0	0	0	0
24	132	0	0	0	0	0	0
112	0	0	0	0	0	0	0
28	-34	0	0	0	0	0	0
18	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Reconstructed
DCT coefficients

165	159	149	136	123	112	104	100
148	141	130	115	101	89	80	75
125	119	108	95	81	70	62	58
104	100	94	86	79	73	69	67
86	86	85	86	87	89	91	93
78	80	83	89	97	104	111	115
85	88	92	99	107	116	123	127
99	100	104	109	116	123	130	133

Reconstructed
pixel values

- Difference between original block and reconstructed block is the error
- Largest magnitude pixel error is 10
- Mean squared error is 20.3

-5	-4	2	4	10	8	-4	-5
1	3	-9	-8	-2	0	0	3
-3	-5	-5	4	3	-5	-2	-3
6	1	0	4	6	7	1	8
4	-1	-5	-1	3	-4	4	7
4	7	7	6	4	4	1	4
4	2	-2	2	-1	-7	-3	-2
-4	0	0	0	3	5	-1	8

Reconstruction
Error

JPEG: Controlling the rate

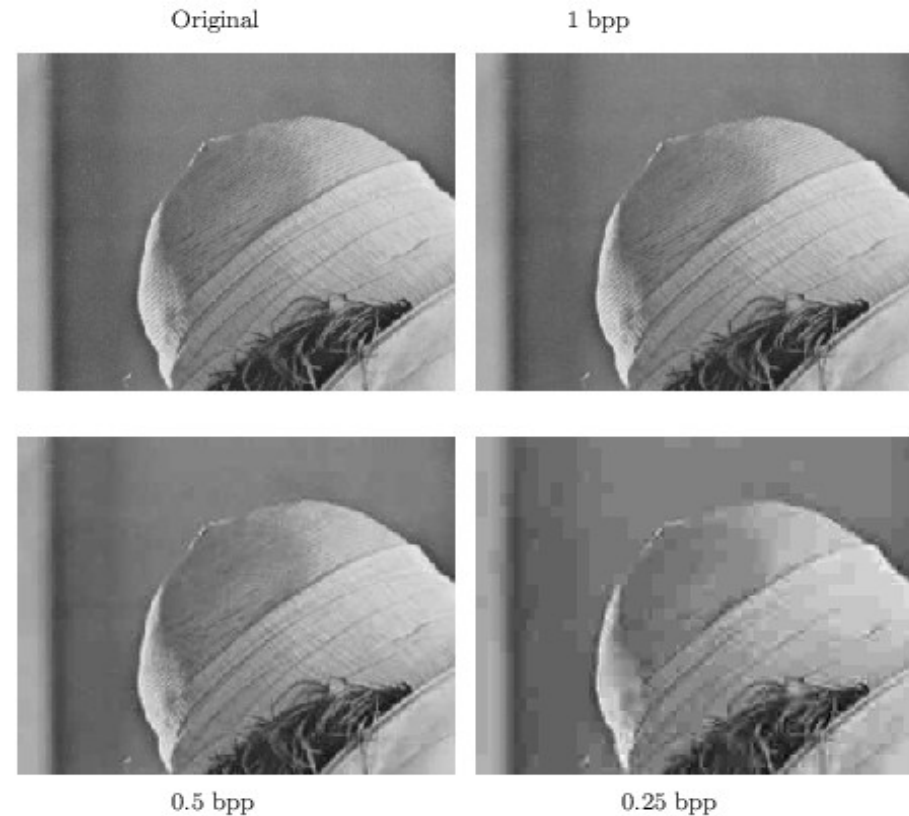
If you use the recommended quantization table... you get only one possible rate & quality.

- What if you want higher quality?
- What if you want lower rate?

Scale the default tables as needed

Called *Quantization Parameter (QP)*

Controls the quality → bit rate comes out however it comes out



JPEG Example



Gonzalez and Woods
Figure 8.29

25:1 compression
about 0.32 bits per pixel

52:1 compression
about 0.15 bits per pixel

JPEG allows one to use different Q Tables and Huffman Tables

Design your own quantization matrix for specific conditions:

- Conditions related to spatial frequency (pixel size, viewing distance, aspect ratio)
- Ambient lighting
- Image content (background luminance, masking)
- Different color spaces
- Optimize for whatever processing comes next (e.g., halftoning and printing)

Design your own Huffman tables:

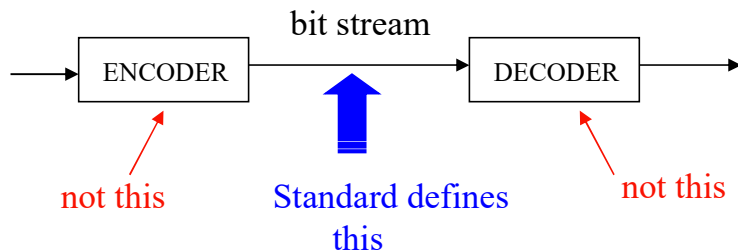
- Tailor Huffman Code for low bit rates
 - Can easily get 10% reduction in bit rate
- Tailor Huffman Code for individual images
 - Need to use a 2-pass technique
 - First pass: transform and quantize the blocks, see what symbol pairs come out
 - Design a Huffman code for that distribution
 - 2nd pass: Do JPEG compression with that Huffman code

Strategy for standardization

- Key purposes of standards are to
 - Facilitate interoperability
 - Maximize support for
 - Current applications
 - As-yet-unknown applications
 - Development of a variety of implementations
- Achieved by focusing standardization on:
 - 1) What is a legal bitstream?
 - Specify rules to construct a legal bitstream (syntax)
 - 2) What should be done with it to generate displayable audio-image-video material?
 - Explain how an idealized decoder could process a legal bitstream

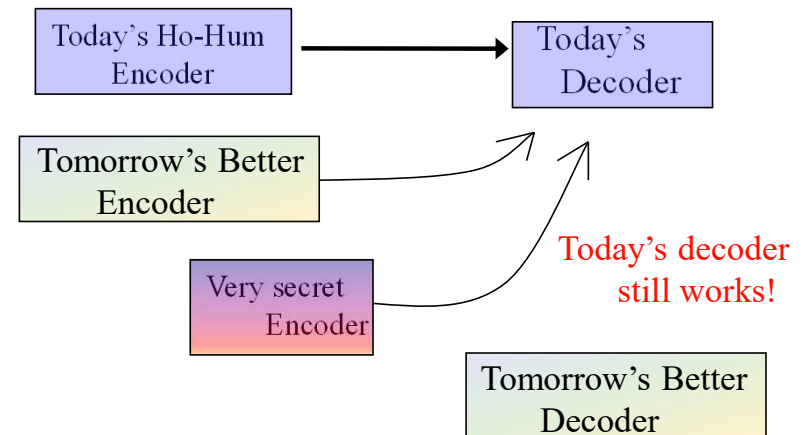
JPEG: What does the standard actually specify?

- The image and video compression standards define syntax for the bit stream between encoder and decoder



- Encoder is not specified by the standard except that it produces a compliant bit stream
- Compliant decoder must interpret all legal bit streams

- This allows future encoders of better performance to remain compatible with existing decoders.
- Also allows for commercially secret encoders to be compatible with standard decoders



Extra Smarts in the Encoder: Example: Lying to the Decoder

Suppose encoder knows some portions of the image are not important

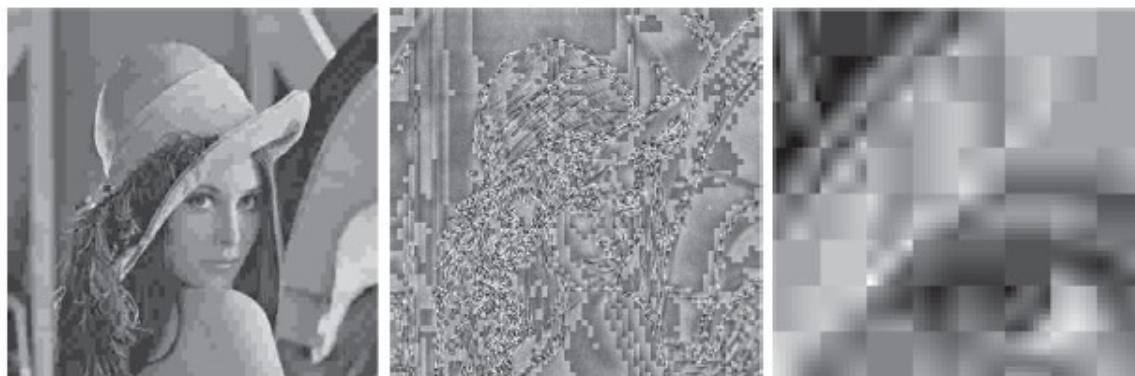


Maybe you don't care much about the suburban part of the picture, or maybe you only care about the suburban part

- Encoder could decide to zero out some coefficients in the unimportant regions
 - Declare EOB even though there are more non-zero coefficients in the block
 - Bit rate will go down
- Decoder will receive a valid bit stream, will not know that it was lied to
- Reconstruction quality has been selectively adjusted
 - Standard does *not* specify how the encoder goes about generating a valid bitstream
 - The encoder can generate a valid bitstream in many ways

Extra Smarts in the Decoder

1. Decoder can smooth out blocking artifacts



Decoder and encoder can both do things beyond what is specified in the standard

2. At very low bit rates, few AC coefficients are transmitted
→ decoder can try to predict some missing AC coefficients



JPEG Modes

What we have seen so far is called **baseline sequential JPEG**

JPEG has several other modes of encoding:

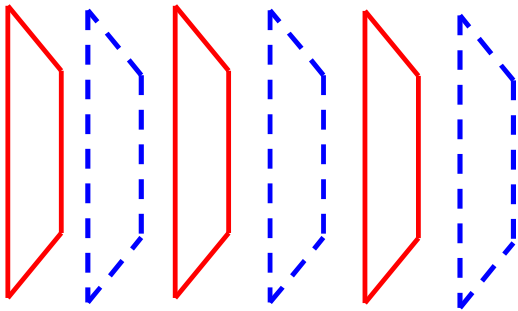
Lossless mode (uses predictive coding and variable length coding, **no** transforms, **no** quantization)

Progressive mode: Image quality improves progressively as more bits arrive

Hierarchical mode: send initially at reduced resolution, more resolution as more bits arrive

Progressive / Layered / Scalable Coding

Concept: The compressed bit stream is composed of *layers*. If the decoder receives and decodes more layers, the image/video quality gets better.



Video: Temporal Scalability. For example, base layer has 15 frames per second, enhancement layer allows display at 30fps



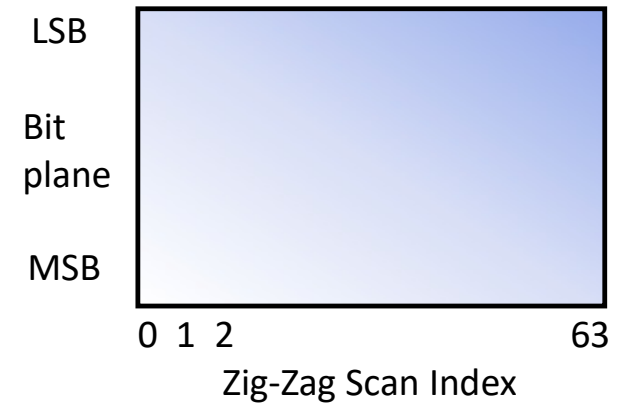
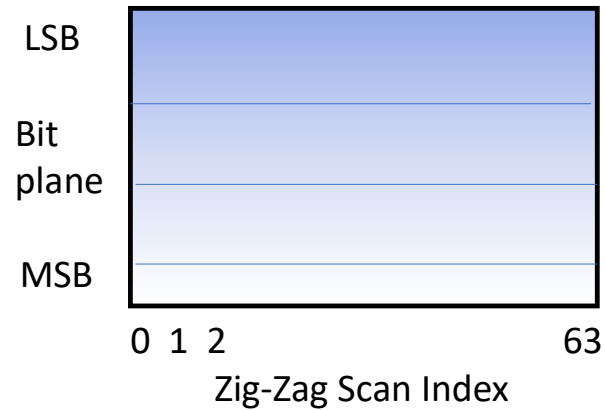
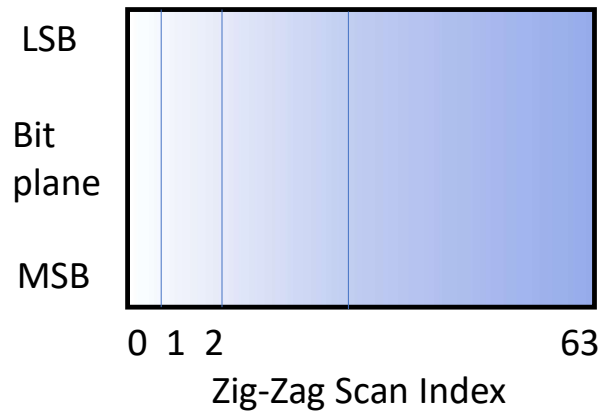
Spatial Scalability. For example, base layer has 256 x 256 image, enhancement layer allows 512 x 512 decoding



Quality Scalability. Base layer has highly compressed version, enhancement layer allows decoding of improved version

Progressive Coding in JPEG

Many ways to do this



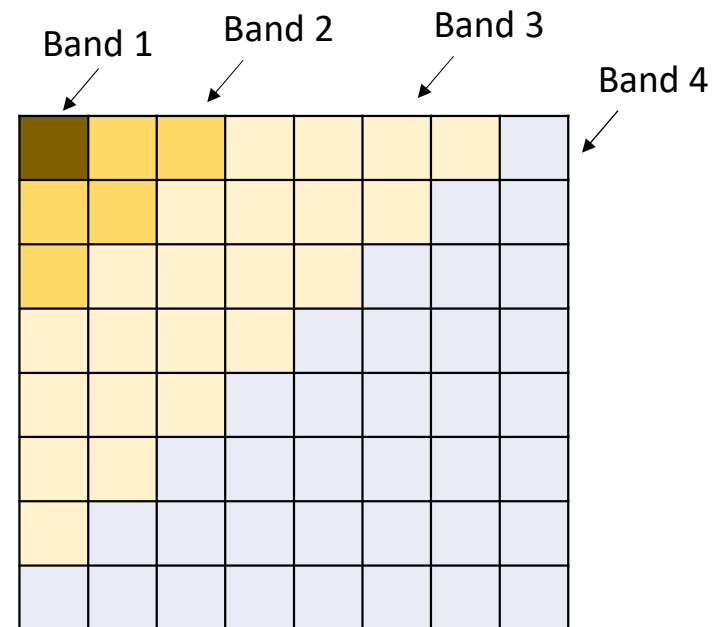
JPEG Progressive Mode: Spectral Selection

Spectral selection:

- Band 1: DC coefficient
- Band 2: AC coefs 1-5
- Band 3: AC coefs 6-27
- Band 4: AC coefs 28-63

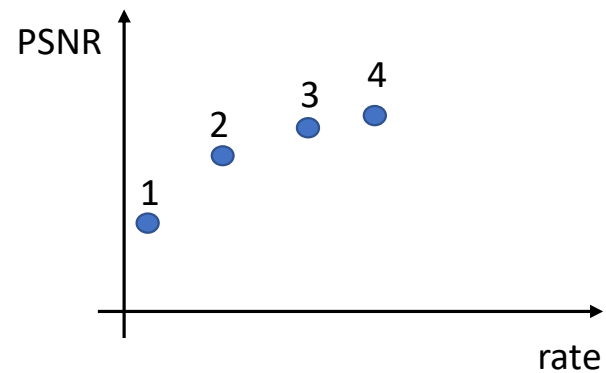
Band 1 is coded first, for the whole image

Then band 2 is coded for the whole image, etc.



JPEG Progressive Mode: Spectral Selection

	1	0	0	0	0	0	0
-2	-1	3	0	0	0	0	
0	3	0	1	0	0		
0	0	0	0	-2			
0	0	-1					
0	0						
0							



What price do you pay for doing this?

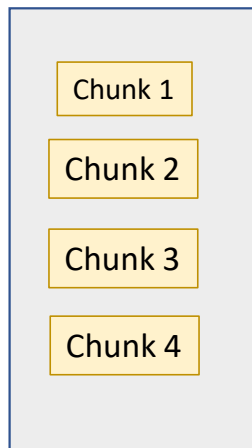
Band 1: DC Coefficient

Band 2: (0,1), (0,-2), (1, -1), EoBand

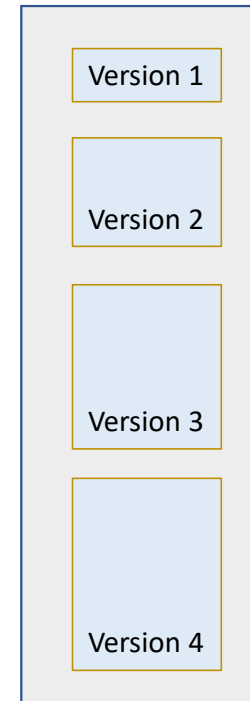
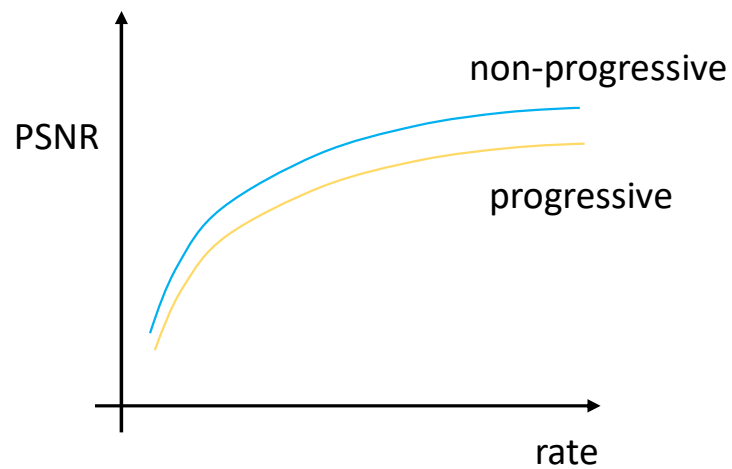
Band 3: (1,3), (0,3), (8,1), (5,-1), EoBand

Band 4: (3, -2), EoB

JPEG Progressive Mode



Progressive
(Layered)
Server



Non-Progressive
Server

JPEG Progressive Mode: Successive Approximation

DCT coefficients are divided by a power of 2 before coding. In the decoder coefficients are multiplied by same power of 2 before IDCT.

The precision of the coefficients is successively increased.

23	9	-1	0
3	2	0	0
-11	-4	0	0
0	0	0	0



Divide by 8
and round

3	1	0	0
0	0	0	0
-1	0	0	0
0	0	0	0

Code: DC = 3
(0,1), (1,-1), EOB
Decoder x8

24	8	0	0
0	0	0	0
-8	0	0	0
0	0	0	0

Original block of DCT step
indexes (Only showing the upper
left 4x4 block-- all the rest in the
8x8 block are 0)

Decoder reconstruction
at this point

JPEG: Successive Approximation

Now consider the division by 4:

23	9	-1	0
3	2	0	0
-11	-4	0	0
0	0	0	0



Divide by 4
and round

6	2	0	0
1	0	0	0
-3	-1	0	0
0	0	0	0

Original block of DCT
step indexes

Decoder already
know this:

24	8	0	0
0	0	0	0
-8	0	0	0
0	0	0	0

For the coefficients where the decoder has
zero history: Code them with runlengths of
zeros (skipping over the coefficients with
non-zero history) plus one bit for the sign:
0, +, 4, -, EOB

		0	0
1	0	0	0
	-1	0	0
0	0	0	0

For each coefficient with non-zero history: Send 1 bit to code the
correction

For example: Decoder knows that $DC \in (20, 28]$ because
 $\text{round}(DC/8) = 3$, so now send 1 bit to say which half it's in:

Send 0 if $DC \in (20, 24]$ and send 1 if $DC \in (24, 28]$

After this
pass, decoder
knows this:

22	10	0	0
4	0	0	0
-10	-4	0	0
0	0	0	0